

Proven C Book

proven 라이브러리에 기반한 모던 C 입문

rubidus

차례

1	배경설명	6
1.1	프로그래밍이란 무엇인가	6
1.2	C는 어디에 있는가	6
1.3	지금 C의 세계는 요동치고 있다	6
1.4	왜 이 책을 만들었나	7
1.5	이 책을 읽는 법	8
2	단순한 기계 모델 ↔ C의 탄생	10
2.1	컴퓨터를 세 부품으로 그리기	10
2.2	비트 — 정보의 최소 단위	11
2.3	그 단순한 기계의 시절, C가 태어났다	12
3	워드와 주소 ↔ C의 원형	14
3.1	사물함 복도	14
3.2	워드 — 기계의 자연스러운 한 움큼	14
3.3	엔디안 — 여러 칸에 수를 넣는 두 가지 순서	15
3.4	C의 원형이 여기 있다	16
4	주소의 특수 지식 — 0번지, 정렬, 하위 비트	17
4.1	0번 사물함에는 무엇이 있는가	17
4.2	널 삼형제 — 이름만 닮은 세 가지	18
4.3	정렬 — 두 칸짜리 짐은 아무 데나 못 넣는다	18
4.4	하위 비트의 재주 — 태그 포인터	18
5	정수의 표현 — 부호, 오버플로, 시프트	20
5.1	부호 없는 정수 — 시계처럼 도는 수	20
5.2	음수를 담는 세 가지 약속	20
5.3	세 약속의 경쟁, 그리고 C23의 결단	21
5.4	시프트 — 비트를 통째로 밀기	22
5.5	부호 확장 — 좁은 그릇에서 넓은 그릇으로	23
6	수의 표현 ↔ IEEE 754라는 계약	25
6.1	고정소수점 — 소수점을 약속으로 못박다	25
6.2	부동소수점 — 소수점을 데이터에 싣다	25
6.3	근사가 일으키는 세 가지 사건	26
6.4	난립, 그리고 IEEE 754라는 계약	27
7	문자와 텍스트 ↔ 표준 속의 흉터	29
7.1	문자는 번호다	29
7.2	ASCII와 EBCDIC — 두 개의 표	29
7.3	ISO 646 — 국가 변형과 C의 삼중자	29
7.4	8비트의 백가쟁명, 그리고 한글	30
7.5	유니코드와 UTF-8 — 하나의 표, 영리한 담기	30
8	스트림의 기원 ↔ 펀치카드, 라인프린터, 프린터 터미널	32
8.1	펀치카드 — 한 장이 한 줄이다	32
8.2	라인프린터 — 줄 단위로 찍는 출력	32
8.3	프린터 터미널 — 종이 위의 대화, 그리고 tty	32
8.4	화면 터미널, 그리고 스트림이라는 유산	33
9	기억의 분화 — 레지스터, 캐시, 다층의 사다리	34

9.1	고백 — 그 그림은 과하게 단순했다	34
9.2	레지스터 — 원래부터 있던, CPU의 손안	34
9.3	벌어지는 격차 — 레지스터와 메모리 사이	34
9.4	캐시 — 격차의 틈에 끼어든 중간층	35
9.5	캐시의 분화 — 다층의 사다리	35
10	속도의 기계장치 ↔ 표준의 탄생	37
10.1	파이프라인 — 조립 라인 위의 명령들	37
10.2	분기 예측 — 갈림길을 미리 짐작하다	37
10.3	클록의 한계, 그리고 멀티코어	38
10.4	같은 시대, C는 계약서를 썼다 — 표준 이전의 C와 C89	38
11	컴파일러 최적화 ↔ 추상 기계	40
11.1	편집자의 작업실 — 컴파일러가 하는 일	40
11.2	추상 기계 — 계약서 속의 가상 기계	40
11.3	계약의 정밀화 — C99, C11	41
12	그래서 C는 추상적인 언어다	43
12.1	논제의 완성 — 세 조각을 합치다	43
12.2	요즘의 흐름 — 포인터에 붙는 출처	43
12.3	마무리 파노라마 — C는 어느 기계의 언어인가	44
12.4	제2부를 닫으며	44
13	헬로 월드	46
13.1	첫 프로그램	46
13.2	한 줄씩 읽기	46
14	일반적인 컴파일 과정	47
14.1	겉보기 — 명령 한 줄	47
14.2	1주자 — 전처리: 텍스트를 짜깁기한다	47
14.3	2주자 — 컴파일: C를 어셈블리로	47
14.4	3주자 — 어셈블: 기계 명령으로 찍어 내다	48
14.5	4주자 — 링크: 조각을 잇고 구멍을 메운다	48
15	개발환경 구축	49
15.1	필요한 것은 셋뿐이다	49
15.2	경고 — 공짜 검토를 켜 두어라	50
15.3	새니타이저 — 실행 중에 치는 그물	50
15.4	제3부를 닫으며	51
16	프로그램의 구조	53
17	수식 — 값이 되는 것	54
18	함수 사용 — 부르는 법	55
19	출력	56
20	변수 선언	58
21	함수 선언과 정의	59
22	입력	60
23	정수 — 유한한 수의 세계	62
24	정수의 연산 — 나눗셈, 비트	63
25	불리언과 비교	64
26	결정 — if와 switch	65

27 반복 — 루프와 불변식	66
28 함수의 의미 — 값 복사와 부수효과	67
29 객체, 주소, 포인터	69
30 널 — 삼형제의 정식 취급	70
31 포인터의 규칙 — 정렬과 프로버넌스	71
32 배열	72
33 문자열	73
34 안전한 입력과 proven의 등장	74
35 수명과 저장 기간	75
36 동적 메모리	76
37 구조체	78
38 공용체와 표현	79
39 실수 — 근사의 수학	81
40 오류와 계약	82
41 정의되지 않은 동작	83
42 여러 파일 — 분할과 링크	85
43 표준 라이브러리의 지형	86
44 proven 전면	87
45 모던 C 총정리	88

제1부 — 바탕

1 배경설명

이 장이 끝나면

프로그래밍이 무엇인지, C가 어떤 언어이고 지금 어떤 처지에 있는지, 그리고 왜 하필 지금 새로운 C 입문서가 필요한지 알게 된다. 이 책이 어떤 방식으로 쓰였고 어떻게 읽으면 되는지도 여기서 정한다.

1.1 프로그래밍이란 무엇인가

컴퓨터는 시키는 일을 하는 기계다. 문제는 시키는 방법이다. 컴퓨터는 사람의 말을 알아듣지 못하고, 지독하게 단순한 지시를 순서대로 받을 때만 움직인다. **프로그래밍**이란 우리가 원하는 일을 그 단순한 지시들의 목록으로 바꾸어 적는 일이고, 그 목록을 적는 약속된 표기법이 **프로그래밍 언어**다.

언어는 수백 가지가 있다. 그중 어떤 언어는 사람 쪽에 바짝 붙어 있어서 쓰기 편하고, 어떤 언어는 기계 쪽에 바짝 붙어 있어서 기계를 속속들이 다룰 수 있다. 이 책이 다루는 C는 후자의 대표다 — 반세기 동안 기계 쪽 자리를 지켜 온, 시스템의 언어다.

1.2 C는 어디에 있는가

C는 눈에 잘 띄지 않는다. 화려한 앱과 웹사이트의 얼굴에는 다른 언어들이 서 있다. 그러나 그 얼굴들 밑을 들추면 거의 어디에나 C가 있다.

● 오늘 하루가 C 위에서 흘러간다

아침에 스마트폰 알람이 울린다 — 그 운영체제의 핵심(커널)은 C로 쓰여 있다. 메시지를 보내면 통신 모듈의 펌웨어가 움직인다 — C다. 웹을 열면 서버와 브라우저가 데이터를 주고받는다 — 그 암호화 라이브러리와 통신 도구(OpenSSL, curl)가 C다. 앱이 무언가를 저장한다 — 세계에서 가장 널리 깔린 데이터베이스 SQLite가 C다. 자동차의 제동 장치, 세탁기의 제어판, 심박 조율기 — 임베디드 기기의 대부분이 C다. 파이썬으로 인공 지능을 돌려도, 그 밑에서 실제 계산을 하는 엔진의 상당 부분이 C(와 그 주변 언어)로 되어 있다. C를 배운다는 것은 이 보이지 않는 층을 읽고 쓸 수 있게 된다는 뜻이다.

1.3 지금 C의 세계는 요동치고 있다

그런데 왜 “지금” 새 입문서인가. 오래된 언어라면 오래된 책으로 배워도 되지 않는가. 그렇지 않다 — 지금 C를 둘러싼 풍경은 조용해 보여도 크게 요동치는 중이고, 이 요동이 이 책의 출발점이다.

첫째, C 안에서 여러 시대가 병립하고 있다. C는 표준이라는 공식 규격으로 관리되는 언어이고, C89(1989), C99, C11, C17을 거쳐 최신판 C23까지 왔다. 문제는 옛 표준이 은퇴하지 않는다는 것이다. 산업 현장에는 C89 시절 관행 그대로의 코드가 여전히 산더미처럼 살아 있고, 많은 교재가 그 시절 어법으로 쓰여 있으며, 그 곁에서 C23은 `bool`과 `nullptr` 같은 새 어휘를 표준에 들였다. 같은 “C”라는 이름 아래 서른 해 넘는 시차의 방언들이 공존한다.

문 옛 표준의 코드가 그렇게 많다면, 옛 어법부터 배우는 것이 실용적이지 않은가?

답 읽는 것과 쓰는 것을 나누면 답이 선다. 옛 코드를 읽을 일은 언젠가 생긴다 — 그래서 이 책도 역사와 옛 어법의 사연을 곳곳에서 다룬다. 그러나 새로 쓰는 코드가 옛 어법일 이유는 없다. 새 표준은 옛 표준의 함정을 수십 년 치 실전 경험으로 메운 결과물이다. 처음 배우는 사람일수록 가장 안전해진 오늘의 어법으로 시작하는 것이 이득이고, 옛 어법은 “왜 저렇게 썼는지”를 아는 교양으로 충분하다.

둘째, 이웃 언어들이 C의 영역을 잠식하고 있다. 한 지붕에서 출발한 C++은 빠른 주기로 판을 거듭하며 거대한 언어가 됐다 — 표현력은 강력해 졌지만, 언어 전체를 아는 사람이 드물다는 말이 나올 만큼 덩치가 불었다. 다른 쪽에서는 새 세대의 시스템 언어들이 나타났다. Rust는 C가 오래 겪어 온 메모리 사고를 언어 차원에서 봉쇄하겠다는 약속으로, Zig는 “C를 계승하되 더 단순하고 정직하게”라는 구호로, 각각 C가 지키던 자리를 파고 들고 있다. 운영체제 커널에 Rust 코드가 들어가기 시작한 것은 상징적인 사건이다.

셋째, 그 압박 속에서 C 자신도 변화의 조짐을 보이고 있다. C23은 근래 가장 큰 폭의 개정이었고, 표준 위원회는 포인터의 규칙을 더 엄밀하게 다듬는 작업(프로버넌스)과 안전성 논의를 이어 가고 있다. C는 느리게 움직이는 언어지만, 방향은 분명하다 — 더 명확한 계약, 더 안전한 기본값 쪽이다.

△ “C는 낡은 언어라서, 배워도 곧 쓸모없어진다”

그렇듯한 걱정이다 — 새 언어들이 저렇게 몰려오고 있으니. 그러나 현실의 수치는 반대를 가리킨다. 위에서 본 것처럼 세상의 기반 시설이 C 위에서 있고, 이 층은 교체 비용이 너무 커서 수십 년 단위로만 움직인다. 새 언어들조차 C와 대화하는 능력(C 인터페이스)을 생존 조건으로 갖추고 태어난다 — C는 시스템 세계의 공용어라서, Rust를 쓰는 Zig를 쓰는 결국 C를 읽고 이해해야 한다. 낡기는커녕, C는 이웃들이 늘어날수록 더 중요해지는 종류의 언어다.

문 그러면 차라리 Rust나 Zig부터 배우는 것이 낫지 않은가?

답 정직하게 답하면, 목적에 따라 다르다. 당장 안전한 응용 프로그램을 하나 만드는 것이 목표라면 다른 선택지도 좋다. 그러나 컴퓨터가 실제로 어떻게 움직이는지를 바닥부터 이해하는 것이 목표라면 C만 한 교재가 없다. C는 기계를 가장 얇게 감싼 언어라서, C를 배우는 과정이 곧 기계와 메모리와 컴파일러를 배우는 과정이 된다. 그리고 그 이해는 Rust로 가든 Zig로 가든 그대로 이고 갈 수 있는 밑천이다. 이 책은 그 밑천을 오늘의 어법으로 마련해 주는 책이다.

1.4 왜 이 책을 만들었나

이 요동치는 풍경이 이 책의 첫 번째 이유다. 여러 표준이 병립하고, 이웃 언어들이 경쟁을 걸어오고, C 자신이 변하고 있는 지금 — 옛 어법의 관성으로 쓰인 입문서가 아니라, **오늘의 C(C23)와 오늘의 관행에서 출발하는** 입문서가 필요하다고 판단했다. 그래서 이 책은 목차부터 다시 짰다. 문법 항목을 순서대로 나열하는 대신, 컴퓨터라는 기계의 기본부터 시작해 개념이 필요해지는 순서대로 C를 만난다.

두 번째 이유는 밝혀 두는 것이 정직하겠다. 저자는 **proven**이라는 C 라이브러리를 만들어 공개하고 있다. proven은 C 프로그래밍에서 가장 사고가 잦은 기본기 — 문자열 다루기, 입력 처리, 수의 변환 같은 일 — 를 검증된 형태로 제공하는 것을 목표로 하는 라이브러리다. C의 유명한 함정들은 대부분 이 기본기 층에서 터지는데, 표준 라이브러리는 역사적 이유로 그 함정을 그대로 안고 있다. 이 책의 뒷부분(제7부부터)에서는 그 함정이 실제로 왜 위험한지 보인 다음, proven이 그것을 어떻게 막는지 예제의 기반으로 사용한다. 이 책은 그 라이브러리를 알리려는 목적도 겸해서 만들어졌다 — 다만 순서는 지킨다. 먼저 표준 C만으로 개념을 온전히 배우고, proven은 그 필요가 스스로 증명된 시점에 등장한다.

문 라이브러리를 홍보하는 책이라면, 내용이 그쪽으로 치우치지 않는가?

답 치우치지 않도록 구조로 막아 두었다. 이 책의 앞 30여 개 장은 proven을 한 줄도 쓰지 않는다 — 순수하게 컴퓨터의 기본과 표준 C만으로 진행된다. proven이 등장하는 것은 “왜 이런 도구가 필요

한가”가 본문 속에서 스스로 드러난 뒤이고, 그때도 표준적인 방법을 먼저 보인 다음 비교로 제시한다. proven 없이도 이 책의 C 지식은 온전히 성립한다.

1.5 이 책을 읽는 법

이 책은 읽는 책이다. 당연한 말 같지만, 프로그래밍 책으로서는 드문 약속이다.

시키지 않는다. 이 책에는 연습문제가 없고, “직접 해 보라”는 지시도 없다. 컴퓨터 앞에 앉아 있지 않아도 된다 — 소파에서, 지하철에서, 그냥 소설처럼 앞에서 뒤로 읽으면 된다. 모든 코드는 지면 위에서 저자가 처음부터 끝까지 시연하고, 실행 결과까지 지면에 인쇄되어 있다. 그 실행 결과는 장식이 아니라 실제 결과다 — 이 책에 실린 모든 예제는 기계가 실제로 컴파일하고 실행해서 얻은 출력을 그대로 실는다.

문답으로 진행한다. 본문 곳곳에서 방금 읽은 내용에 대해 독자가 품었을 법한 질문을 그 자리에서 묻고, 바로 답한다. 질문을 만난 순간 잠깐 스스로 답을 떠올려 보고 다음 줄을 읽으면 가장 좋지만, 그냥 내리읽어도 된다 — 문답 자체가 설명의 일부다.

외우게 하지 않는다. 기억할 가치가 있는 것은 뒤 장의 서두에서 “돌아 보기”라는 이름으로 다시 찾아온다. 이전 장의 내용을 조금 더 깊은 질문으로 다시 만나게 되는데, 이것이 이 책의 복습 장치다. 잊었어도 괜찮다 — 답이 바로 옆에 있다.

작은 장으로 빠르게 간다. 장 하나는 짧다. 한 번에 하나의 착상만 다루고 다음으로 넘어간다. 큰 주제는 여러 장에 걸쳐 나선처럼 여러 번, 매번 조금 더 깊게 지나간다. 한 장에서 모든 것을 말하지 않으니, 어딘가 궁금증이 남았다면 그것은 대개 의도된 것이다 — 몇 장 뒤에서 그 궁금증을 다시 만나게 된다.

이제 시작한다. 첫걸음은 C가 아니라 컴퓨터 그 자체다 — C가 왜 이렇게 생겼는지는, C가 태어난 기계를 보아야 이해되기 때문이다. 다음 부에서 컴퓨터를 아주 단순한 그림으로 그리는 것부터 시작한다.

제2부 — 전산의 기본과 배경지식

이 부는 C 문법을 하나도 가르치지 않는다. 대신 이후의 모든 장이 딛고 설 바닥을 깎는다. 세 걸음으로 오른다.

기계와 기억 (2-4장) — 컴퓨터를 세 부품의 단순한 그림으로 그리고, 기억이라는 사물함 복도를 걷는다. 손에 잡히는 이야기다.

표현의 사다리 (5-8장) — 그 사물함에 정수를, 소수점 있는 수를, 글자를, 그리고 글자의 흐름을 담는 법. 한 단씩 오른다.

속도와 추상 (9-12장) — 기계가 빨라지며 기억이 어떻게 갈라졌는지 (레지스터·캐시), 속도의 기계장치들, 컴파일러라는 편집자, 그래서 C가 왜 “추상적인 언어”인지. 이 부의 결론이 여기서 완성된다.

각 걸음마다 C의 역사가 나란히 걷는다. 연대와 인명을 외우게 하려는 것이 아니다 — 오늘의 C가 왜 이런 모양인지, 역사가 가장 짧은 설명이기 때문이다.

2 단순한 기계 모델 ↔ C의 탄생

이 장이 끝나면

컴퓨터를 세 가지 부품 — 계산하는 곳, 기억하는 곳, 박자를 맞추는 것 — 의 단순한 그림으로 그릴 수 있게 된다. 정보의 최소 단위인 비트가 무엇인지, 왜 하필 2진법인지 알게 된다. 그리고 그 단순한 기계의 시절에 C라는 언어가 어떻게 태어났는지 보게 된다.

돌아보기 1장에서 C가 운영체제와 임베디드 기기, 인터넷 기반시설 속에 지금도 살아 있다고 했다. 반세기가 넘은 언어가 어떻게 아직 그 자리에 있는가?

답 C가 서 있는 자리가 특별하기 때문이다. 그 자리는 기계와 사람이 만나는 가장 낮은 층이고, 기계는 반세기 동안 빨라졌을 뿐 일하는 방식의 뼈대는 놀랄 만큼 그대로다. 그 뼈대를 이 장에서 직접 들여다본다. 뼈대가 그대로인 한, 뼈대의 언어도 그대로 남는다.

2.1 컴퓨터를 세 부품으로 그리기

컴퓨터는 복잡한 물건이다. 그러나 복잡한 것을 이해하는 첫걸음은 언제나 과감하게 단순한 그림을 그리는 것이다. 지금부터 컴퓨터를 딱 세 부품으로 그린다.

첫째, **CPU**. 계산하는 곳이다. 더하고, 비교하고, 다음에 무엇을 할지 정한다.

둘째, **메모리**. 기억하는 곳이다. 계산에 쓸 재료와 계산의 결과가 여기에 놓인다.

셋째, **클럭**. 박자를 맞추는 것이다. 메트로놈처럼 일정한 간격으로 똑딱이고, CPU는 그 박자에 맞춰 한 걸음씩 일한다.

이 그림에서 컴퓨터가 하는 일은 이것이 전부다: **클럭이 똑딱일 때마다, CPU가 메모리에서 다음 할 일을 하나 가져와서, 그대로 한다.** 이것을 끝없이 반복한다.

문 “1초에 40억 번” 같은 광고 문구의 기가헤르츠(GHz)가 이 클럭인가?

답 그렇다. 4GHz는 메트로놈이 1초에 40억 번 똑딱인다는 뜻이다. 박자마다 CPU가 일을 한 걸음씩 진행하니, 박자가 빠를수록 같은 시간에 더 많은 걸음을 걷는다. 다만 걸음 수가 전부는 아니라는 것을 10장에서 보게 된다 — 한 걸음에 여러 일을 하는 요령이 현대 CPU의 진짜 무기다.

문 겨우 “가져와서 그대로 한다”의 반복으로 게임과 동영상과 인공지능이 어떻게 가능한가?

답 벽돌 한 장은 단순하지만 벽돌로 성을 쌓을 수 있는 것과 같다. 단순한 걸음이라도 1초에 수십억 번이면, 그리고 그 걸음들을 겹겹이 조직하면 — 작은 일들이 모여 함수가 되고, 함수가 모여 프로그램이 되고, 프로그램이 모여 운영체제가 되면 — 어떤 복잡한 일이든 지을 수 있다. 이 “겹겹이 쌓기”가 전산학의 심장인 **추상화**이고, 이 책 전체가 그 층계를 오르는 이야기다.

CPU가 “다음 할 일”로 가져오는 것을 **명령**이라 부른다. 명령은 대단한 것이 아니다. “이 수와 저 수를 더 해라”, “이 값을 저 칸에 넣어라”, “방금 결과가 0이면 다른 곳으로 건너뛰어라” — 이 정도 수준의, 지독하게 단순한 지시다. 프로그램이란 이런 명령을 순서대로 늘어놓은 목록이고, 프로그래밍이란 그 목록을 사람이 감당할 수 있는 방식으로 적는 일이다.

2.2 비트 — 정보의 최소 단위

메모리가 “기억하는 곳”이라 했다. 그러면 무엇을, 어떤 모양으로 기억하는가.

전자 회로가 가장 잘하는 일은 두 가지 상태를 구별하는 것이다. 전압이 높거나, 낮거나. 스위치가 켜져 있거나, 꺼져 있거나. 이 둘 중 하나를 담는 가장 작은 기억 단위를 **비트(bit)**라 부른다. 우리는 두 상태에 0과 1이라는 이름을 붙인다.

△ “컴퓨터 안 어딘가에 0과 1이 글자처럼 적혀 있다”

그럴듯한 생각이다 — 책과 화면마다 컴퓨터는 0과 1로 가득하다고 그려지니까. 그러나 기계 안에 숫자 글자가 있는 것이 아니다. 실제로 있는 것은 높거나 낮은 전압, 차 있거나 빈 전하 같은 물리 상태뿐이다. 0과 1은 그 물리 상태를 사람이 읽기 위해 붙인 **이름**, 즉 해석이다. 이 구별은 한가한 트집이 아니다 — “기억된 것 자체”와 “그것을 어떻게 읽을 것인가”의 분리는 이 책에서 계속 되돌아올 주제이고, 나중에 같은 비트 열을 수로도 글자로도 읽게 되는 이유다(6장, 7장).

문 왜 하필 두 상태인가? 스위치 하나에 0부터 9까지 열 가지 상태를 담으면 훨씬 효율적이지 않은가?

답 실제로 초기에는 10진 컴퓨터가 만들어지기도 했다. 그러나 열 단계의 전압을 구별하려면 단계 사이 간격이 좁아지고, 간격이 좁으면 잡음이나 온도 변화에 이웃 단계로 넘어가 버린다 — 즉 틀리기 쉽다. 두 상태는 간격이 가장 넓어서, 회로가 싸고 빠르고 잡음에 강하다. 2진법은 수학적 우아함 때문이 아니라 **공학적 정직함** 때문에 이겼다.

비트 하나는 두 가지를 구별한다. 비트를 여러 개 묶으면 구별할 수 있는 가짓수가 곱으로 불어난다. 두 개면 네 가지(00, 01, 10, 11), 세 개면 여덟 가지다. 여덟 개를 묶은 것에는 **바이트(byte)**라는 이름이 붙어 있고, 한 바이트는 256가지를 구별한다.

문 바이트는 왜 하필 여덟 개 묶음인가? 처음부터, 그리고 어디서나 여덟 개였는가?

답 아니다 — 바이트의 크기는 8비트로 고정된 적이 없다. 원래 “바이트”는 **문자 하나를 담는 묶음**이라는 뜻으로 쓰인 말이고, 그 크기는 기계마다 달랐다. 초기 컴퓨터들에는 6비트, 7비트, 9비트 바이트가 실제로 있었고, 워드가 36비트라서 바이트를 6비트씩 여섯 개로 자르던 기계도 있었다. 8비트가 대세가 된 것은 1960년대 IBM의 대형 컴퓨터(System/360)가 8비트를 채택하고 산업이 그 뒤를 따르면서다. 그래서 정확함이 생명인 통신 규격들은 지금도 “바이트” 대신 **옥텟(octet, 정확히 8비트)**이라는 말을 쓴다 — 바이트라는 말만으로는 크기가 보장되지 않았던 역사의 흔적이다.

그리고 이것은 옛날이야기만이 아니다. 지금도 일부 신호 처리용 프로세서(DSP)는 가장 작은 기억 단위가 16비트나 32비트라서, 그 위의 C에서는 바이트가 16비트·32비트다. 흔히 만나는 기계에서는 사실상 전부 8비트지만, “바이트 = 무조건 8비트”는 약속이 아니라 관행이다.

C 표준(C23)의 정의도 그래서 신중하다. C23에서 **바이트**란 “기본 문자 하나를 담을 수 있는, 주소를 붙일 수 있는 가장 작은 기억 단위”이고, 그 비트 수는 `CHAR_BIT`라는 이름으로 공개하되 **8 이상**이라고만 못박는다 — 정확히 8이 아니라 “최소 8”이다. 이 책은 이후 흔한 기계들을 따라 바이트 = 8비트로 놓고 진행하되, 그것이 표준의 보장이 아니라 사실상의 관행임을 여기 적어 둔다. (표준 위원회는 차기 표준에서 8비트로 아예 못박는 방향을 논의하고 있다 — 관행이 반세기를 버티면 약속으로 승격되는 셈이다.)

Σ 묶음의 크기와 가짓수

비트 n 개의 묶음이 구별할 수 있는 상태의 가짓수는, 각 자리가 독립적으로 두 값을 가지므로

$$2 \times 2 \times \cdots \times 2 = 2^n$$

이다. $2^8 = 256$, $2^{16} = 65536$, $2^{32} \approx 4.3 \times 10^9$. 이 책에서 2^n 은 계속 나타난다 — 기억의 크기, 표현 가능한 수의 범위, 주소의 개수가 전부 이 꼴이다.

묶음으로 수를 나타내는 방법은 우리가 쓰는 십진법과 같은 원리인 위치 기수법이다. 십진수 $304 = 3 \cdot 10^2 + 0 \cdot 10^1 + 4 \cdot 10^0$ 이듯, 2진수 $101_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 5$ 다. 자리 하나가 10 대신 2의 거듭제곱을 짊어질 뿐이다.

문 256가지밖에 안 되는 바이트로 어떻게 큰 수를 다루는가?

답 십진법에서 한 자리로 부족하면 자리를 늘리듯, 바이트도 여러 개를 이어 붙여 큰 수를 담는다. 몇 개를 이어 붙이는 것이 자연스러운가는 기계마다 정해져 있다 — 그 “자연스러운 묶음”이 다음 장의 주인공인 워드다.

2.3 그 단순한 기계의 시절, C가 태어났다

지금까지 그린 세 부품의 그림은 오늘의 컴퓨터에게는 지나치게 단순한 그림이다(그 이야기가 9·10장이자). 그런데 1970년대 초의 컴퓨터에게는 이 그림이 거의 사실 그대로였다. 클록은 초당 수십만 번 수준이었고, CPU는 정말로 한 박자에 한 걸음씩 걸었고, 메모리는 정말로 한 줄로 늘어진 사물함이었다.

C는 바로 그 시절, 그 단순한 기계 위에서 태어났다 — 그런데 무(無)에서 솟은 것이 아니라, 짧고 뚜렷한 족보의 끝에서 태어났다. 그 족보를 알면 C의 생김새 몇 가지가 단번에 설명되므로, 여기 간추려 둔다.

출발점은 1960년대 영국의 CPL이라는 야심찬 언어였다 — 너무 야심차서 당대 기계로는 완성이 어려웠고, 케임브리지의 마틴 리처즈가 그 정수만 추려 BCPL(1967)이라는 작고 실용적인 언어를 만들었다. BCPL의 급진적 단순함은 이것이다: **타입이 하나뿐이다**. 모든 값은 그냥 워드 한 칸이고, 그것을 수로 쓸지 주소로 쓸지는 연산이 정한다. 3장에서 배운 “칸에 든 것은 그저 비트이고 해석이 정한다”를 언어 전체로 밀어붙인 셈이다.

벨 연구소의 켄 톰프슨은 낡은 PDP-7 위에서 초기 Unix를 만들며, BCPL을 자기 취향으로 더 줄인 B(1969년경)를 만들었다 — 여전히 타입은 하나였고, 오늘날까지 살아남은 간결한 어법들(+= 같은)이 이 손질에서 왔다. 그런데 연구소가 새 기계 PDP-11을 들이자 B의 전제가 깨졌다. PDP-11의 메모리는 워드가 아니라 **바이트** 단위로 주소가 붙었고(3장의 복도가 바로 이 모습이다), 크기가 다른 데이터 — 1바이트짜리 문자, 2바이트짜리 정수, 그리고 부동소수점 — 를 구별해 다뤄야 했다. “모든 것이 워드 한 칸”인 B로는 이 기계를 제대로 부릴 수 없었던 것이다.

그래서 데니스 리치가 B에 **타입**을 들였다 — 문자와 정수를 구별하고, 무엇의 주소인지 아는 포인터를 만들고, 구조체를 보탰다. 이 개조가 1971 73년 사이에 쌓여 새 이름을 얻은 것이 C다. 즉 C의 타입 시스템은 철학적 설계가 아니라 **바이트 주소 기계라는 현실**에 대한 응답이었다. C가 뻗속까지 바이트 지향인 것(2장의 바이트, 3장의 복도), 포인터가 언어의 한복판에 있는 것, 그러면서도 타입이 험거운 편인 것(타입 하나 짜리 조상의 유산) — 전부 이 족보의 흔적이다.

덧붙여 두 가지. 오늘의 눈에 익숙한 어법이 처음부터 있었던 것은 아니다 — 이를테면 지금의 +=는 초기 C에서 +=라고 적었고, 몇 해 뒤에야 지금 꼴이 됐다(헛갈림 때문이었다: $x=-1$ 이 “빠기”인지 “대입”인지). 언어는 이렇게 실사용의 상처를 먹으며 다듬어진다. 그리고 이 시절의 C에는 아직 표준도, 규격서도 없었

다 — 언어의 정의는 사실상 “리치의 컴파일러가 받아 주는 것”이었다. 이 험거움이 어떤 소동으로 자라 표준(C89)을 부르게 되는지는 10장에서 잇는다.

그래서 초기의 C는 과장을 조금 보태면 **그 시절 기계의 정직한 별명**이었다. C의 연산 하나가 기계 명령 한두 개에 그대로 대응했고, 프로그래머는 C를 쓰면서도 기계가 무엇을 할지 훤히 내다볼 수 있었다. C가 “기계에 가까운 언어”라는 평판은 여기서 왔다 — 그리고 이 평판이 오늘날 절반만 맞는 말이 된 사연이 이 부의 뒷장들(9-12장)이다.

● 이식성 혁명 — Unix를 C로 다시 쓰다

1973년경 Unix의 심장부가 어셈블리에서 C로 다시 쓰였다. 당시로서는 파격이었다 — 운영체제는 기계어 수준에서 짜야 한다는 것이 상식이었기 때문이다. 효과는 압도적이었다. 어셈블리 Unix는 PDP-11에 묶여 있었지만, C로 쓰인 Unix는 “C 컴파일러만 있으면” 다른 기계로 옮길 수 있었다. 운영체제가 특정 기계의 소유물이 아니게 된 것이다. 오늘날 Linux, Windows, macOS의 커널이 모두 C(또는 그 후예)로 쓰여 있고, 새 프로세서가 나오면 가장 먼저 C 컴파일러부터 이식되는 전통이 여기서 시작됐다.

문 50년 전 기계에 맞춘 언어를 지금 배우는 것이 손해는 아닌가?

답 거꾸로다. 기계의 뼈대 — 메모리, 주소, 바이트, 명령의 순차 실행 — 는 50년 동안 바뀌지 않았고, C는 그 뼈대를 가장 얇게 감싼 언어로 남아 있다. 그래서 C를 배우는 일은 특정 언어의 문법 암기가 아니라 컴퓨터 그 자체를 배우는 일에 가깝다. 이후에 어떤 언어를 쓰게 되든, 그 언어의 밑바닥에는 거의 언제나 C가 깔려 있다.

이 장의 그림을 정리하면 이렇다. 기계는 클록의 박자에 맞춰 메모리에서 명령을 가져와 그대로 실행하는 일을 반복하고, 기억은 두 상태의 최소 단위인 비트로 이루어진다. C는 이 그림이 거의 사실이던 시절에, 이 그림을 언어로 옮겨 태어났다.

다음 장은 세 부품 중 메모리를 자세히 들여다본다. 사물함 복도를 걸으며 칸마다 붙은 번호 — **주소** — 를 읽고, 기계가 한 번에 집는 자연스러운 묶음 — **워드** — 을 재고, 여러 바이트짜리 수를 사물함에 넣는 두 가지 순서 — **엔디안** — 를 구경한다. C의 포인터라는 유명한 개념이 바로 이 복도에서 태어났다.

3 워드와 주소 ↔ C의 원형

이 장이 끝나면

메모리를 번호 붙은 사물함들의 긴 복도로 그릴 수 있게 된다. 그 번호가 주소이고, 기계가 사물함을 한 번에 몇 칸씩 다루는지가 워드다. 여러 칸에 걸치는 수를 넣는 두 가지 순서 — 리틀 엔디안과 빅 엔디안 — 를 그림으로 구별하게 된다.

돌아보기 2장에서 바이트 하나는 256가지밖에 구별하지 못하며, 큰 수는 바이트를 여러 개 이어 붙여 담는다고 했다. 그러면 기계는 이어 붙인 바이트들이 “한 덩어리”라는 것을 어떻게 아는가?

답 모른다 — 이것이 이 장의 출발점이다. 메모리 자체는 어느 칸이 어느 칸과 한 덩어리인지 전혀 기록하지 않는다. 덩어리를 아는 것은 메모리가 아니라 읽는 쪽이다. 프로그램이 “여기서부터 네 칸을 하나의 수로 읽겠다”고 정할 뿐이다. 2장의 오개념 — 기억된 것 자체와 읽는 방법의 분리 — 이 여기서 첫 실전을 치른다.

3.1 사물함 복도

메모리를 그림으로 그리면 이렇다. 긴 복도에 같은 크기의 사물함이 한 줄로 끝없이 이어져 있고, 사물함마다 번호가 붙어 있다. 사물함 한 칸의 크기는 1바이트다. 번호는 0부터 시작해 1씩 늘어난다. 이 번호가 주소(address)다.

01001000	01101001	00100001	00000000	11111111
100	101	102	103	104

칸 안에 든 것은 언제나 그저 비트 여덟 개다. 그 여덟 비트가 수인지, 글자인지, 더 큰 무엇의 조각인지는 칸 자신도 복도도 모른다 — 읽는 쪽의 해석이 정한다.

주소에 관해 첫눈에 안 보이는 중요한 사실이 하나 있다. **주소 자체도 수다**. 사물함 번호는 그냥 정수이고, 정수이므로 그것을 다른 사물함에 넣어 둘 수도 있다. “347번 칸에, 512라는 번호를 적어 둔다” — 이상할 것이 하나도 없다. 이 평범한 착상이 나중에 C에서 가장 유명한 개념인 **포인터**가 된다(29장). 지금은 “주소도 수라서 어디에든 적어 둘 수 있다”까지만 챙기면 된다.

문 복도의 길이, 그러니까 사물함의 개수는 얼마나 되는가?

답 주소를 몇 비트로 적느냐가 정한다. 주소가 n 비트 수라면 사물함은 최대 2^n 개까지 구별할 수 있다(2장의 2^n 이 바로 다시 나왔다). 주소를 32비트로 적던 시절의 복도는 최대 약 43억 칸 — 4GiB — 이었고, 오늘의 64비트 주소는 사실상 다 쓸 수 없을 만큼 긴 복도를 만든다. “32비트 시스템에서 메모리 4GB 한계” 같은 말의 정체가 이것이다.

3.2 워드 — 기계의 자연스러운 한 움큼

사물함은 한 칸씩 있지만, 기계가 일할 때 한 칸씩만 집으면 너무 느리다. 그래서 CPU는 여러 칸을 한 움큼에 집도록 만들어져 있다. 그 움큼의 크기 — 기계가 가장 자연스럽게, 한 번에 다루는 비트 수 — 를 워드(word)라 부른다.

“64비트 컴퓨터”라는 말이 바로 이것이다. 그 기계의 워드가 64비트, 즉 8바이트라는 뜻이다. CPU 안의 임시 보관함(레지스터)이 그 크기이고, 메모리를 오가는 통로(버스)가 그 폭에 맞춰져 있고, 주소도 대

개 그 크기의 수로 다룬다. 워드는 기계의 “손 크기”라 해도 좋다 — 손이 큰 기계는 한 번에 더 큰 수를 집는다.

문 그러면 64비트 기계에서는 모든 것이 8바이트 단위로 저장되는가?

답 아니다. 저장은 여전히 바이트 단위로 자유롭다 — 1바이트짜리 글자도, 4바이트짜리 수도 같은 복도에 산다. 워드는 “기계가 가장 편하게 집는 크기”이지 “모든 것의 크기”가 아니다. 실제로 C에는 크기가 다른 여러 수 타입이 있고(22장), 어떤 크기의 데이터를 어디에 두는 것이 편한가라는 문제는 4장(정렬)에서 다시 만난다.

3.3 엔디안 — 여러 칸에 수를 넣는 두 가지 순서

이제 이 장의 하이라이트다. 4바이트짜리 수 하나를 사물함 네 칸에 넣어야 한다. 수를 16진법으로 12 34 56 78이라는 네 바이트 조각으로 나눴다고 하자(16진법 두 자리가 1바이트다). 100번 칸부터 넣는다면 — 어느 조각을 100번 칸에 넣어야 하는가?

두 학파가 있다. **빅 엔디안(big-endian)**은 큰 자리부터 넣는다. 사람이 종이에 수를 적는 순서 그대로다.

12	34	56	78
100	101	102	103

리틀 엔디안(little-endian)은 작은 자리부터 넣는다. 뒤집힌 것처럼 보이지만, “ i 번째 칸에 i 번째로 작은 자리가 있다”는 나름의 정연한 규칙이다.

78	56	34	12
100	101	102	103

지금 이 글을 읽는 컴퓨터와 스마트폰은 거의 확실히 리틀 엔디안이다(인텔·AMD·대부분의 ARM 운용). 날짜 표기와 똑같은 상황이다 — 2026-08-04라고 적는 나라와 04-08-2026이라고 적는 나라가 있고, 어느 쪽도 틀리지 않았지만, **서로의 편지를 그대로 읽으면 사고가 난다.**

문 사람 눈에는 빅 엔디안이 자연스러운데, 왜 대부분의 기계는 “뒤집힌” 리틀 엔디안을 택했는가? 무슨 장점이 있는가?

답 두 가지 실속 때문이다.

첫째, **시작 칸이 흔들리지 않는다.** 리틀 엔디안에서는 수의 가장 작은 자리가 언제나 시작 주소 그 칸에 있다. 위 그림의 수를 “4바이트 전부”로 읽든 “아래 2바이트만”·“아래 1바이트만”으로 줄여 읽든, 읽기 시작하는 칸은 똑같이 100번이다. 빅 엔디안에서는 읽는 폭을 바꿀 때마다 시작 칸을 옮겨 다시 계산해야 한다. 같은 값을 크기가 다른 눈으로 읽는 일이 잦은 기계에게 “시작 주소 불변”은 꽤 큰 단순함이다.

둘째, **계산의 진행 방향과 맞는다.** 손으로 덧셈을 할 때를 떠올리면 된다 — 일의 자리부터 더하고 받아올림을 위로 넘긴다. 여러 바이트짜리 수를 더하는 기계도 마찬가지로 작은 자리부터 처리하는데, 리틀 엔디안에서는 그것이 “낮은 주소부터 순서대로”와 정확히 일치한다. 초기의 작은 CPU들은 첫 바이트를 가져오자마자 덧셈을 시작할 수 있었고, 이 이점이 초창기 설계(인텔 계열의 조상들)에 스며서 오늘날까지 이어졌다.

빅 엔디안의 장점도 공정하게 적으면 — 덤프가 사람 눈에 그대로 읽히고, 바이트를 앞에서부터 통째로 비교하면 수의 대소 비교와 일치한다. 그래서 통신 규약처럼 “사람과 기계가 함께 들여다보는 자리”에는 빅 엔디안이 남았다. 어느 쪽도 우월하지 않다 — 자주 하는 일이 다를 뿐이다.

△ “메모리를 앞에서부터 읽으면 적힌 순서대로 수가 보인다”

그렇듯하다 — 종이에 적힌 수는 그렇게 읽으니까. 그러나 리틀 엔디안 기계에서 12 34 56 78이라는 수는 메모리에 78 56 34 12 순서로 놓인다. 메모리를 칸 순서대로 들여다보면(디버거나 파일 덤프에서 실제로 이렇게 본다) 수가 “뒤집혀” 보인다. 뒤집힌 것이 아니라, 넣는 순서의 약속이 다를 뿐이다. 기억할 것: 메모리 덤프는 종이에 적은 수가 아니다.

● NUXI — 순서가 낳은 유령

초기 Unix를 다른 회사의 컴퓨터로 이식하던 엔지니어들이 화면에서 기묘한 것을 보았다. UNIX라고 찍혀야 할 곳에 NUXI가 찍힌 것이다. 두 기계의 엔디안이 달라서, 2바이트 묶음 안의 글자 순서가 통째로 뒤집힌 결과였다. 이 사건은 “NUXI 문제”라는 이름으로 엔디안 사고의 대명사가 됐다. 같은 종류의 사고는 지금도 난다 — 파일을 한 기계에서 저장해 다른 기계에서 읽을 때, 네트워크로 수를 주고받을 때, 엔디안을 정하지 않으면 수가 깨진다. 그래서 인터넷 규약은 “통신할 때는 빅 엔디안으로 보낸다”는 **네트워크 바이트 순서**를 합의해 두었다.

문 내 컴퓨터가 리틀 엔디안이라는 것을 프로그램으로 확인할 수 있는가?

답 있다 — 여러 바이트짜리 수를 메모리에 넣고, 그 첫 칸을 1바이트만 따로 읽어 보면 된다. 첫 칸에 작은 자리 조각이 있으면 리틀 엔디안이다. 이 확인을 C로 실제로 해 보이는 것이 38장(공용체)의 시연이다 — 같은 기억을 다른 눈으로 읽는 도구가 그때 생긴다.

3.4 C의 원형이 여기 있다

이 장의 복도 그림은 C라는 언어의 설계도이기도 하다. C가 태어난 시절 (2장), 언어를 설계한 사람들은 기계의 이 모습을 감추는 대신 언어의 개념으로 그대로 들어 올렸다. 메모리는 바이트의 열이라는 것 — C의 모든 데이터는 바이트 단위 크기를 갖는다. 칸마다 주소가 있다는 것 — C에서는 거의 모든 것의 주소를 물을 수 있다. 주소도 수라는 것 — 그 수를 담는 타입(포인터)이 언어의 한복판에 있다. 다른 많은 언어가 “사물함 복도”를 프로그래머에게 보이지 않게 봉인해 둔 것과 대조적이다.

이 정직함은 양날이다. 기계를 훤히 내다보며 일할 수 있는 힘이자, 복도에서 길을 잃으면 그대로 사고가 되는 위험이다. 이 책의 제7부가 그 힘과 위험을 정면으로 다룬다.

다음 장은 이 복도의 구석에 있는 흥미로운 특례들을 구경한다. 0번 사물함에는 무엇이 있는가, 왜 두 칸짜리 짐은 아무 번호에나 못 넣는가(정렬), 그리고 번호의 끝자리가 항상 0이라는 사실을 이용해 몰래 정보를 숨기는 재주(태그 포인터)까지 — 주소의 세계가 생각보다 훨씬 사람 사는 동네 같다는 것을 보게 된다.

4 주소의 특수 지식 — 0번지, 정렬, 하위 비트

이 장이 끝나면

사물함 복도의 구석 지식 세 가지를 챙긴다. 0번 사물함은 왜 특별한지, 왜 큰 짐은 아무 번호에나 못 넣는지(정렬), 그리고 번호의 끝자리에 정보를 숨기는 재주(태그 포인터)까지. “널”이라는 이름의 세 가지 다른 물건도 여기서 처음 구별해 둔다.

돌아보기 3장에서 “주소도 수라서 다른 사물함에 적어 둘 수 있다”고 했다. 그런데 적어 둔 주소가 **아직 없음**을 나타내고 싶으면 — “아무 데도 가리키지 않는다”는 표시는 어떻게 하는가?

답 특별한 값 하나를 “없음”의 표시로 약속하면 된다. 그리고 거의 모든 시스템이 그 표시로 0을 골랐다 — 0번 사물함을 아무도 안 쓰는 칸으로 비워 두고, “0이 적혀 있으면 아무 데도 안 가리키는 것”으로 읽는 약속이다. 이 약속된 값이 **널 포인터(null pointer)**다. 그런데 왜 하필 0번 칸을 비워 둘 수 있었는가 — 그 이야기가 이 장의 첫 절이다.

4.1 0번 사물함에는 무엇이 있는가

답부터 말하면, **기계마다 다르다**. 그리고 그 차이가 재미있다.

데스크톱과 서버, 스마트폰처럼 운영체제가 보호 모드로 도는 환경에서는, 운영체제가 0번지 근처를 일부러 **접근 금지 구역**으로 만들어 둔다. 어떤 프로그램이 0번지를 읽거나 쓰려 하면 그 즉시 붙잡혀 종료된다. 이것은 친절이다 — “없음” 표시(널)를 실수로 진짜 주소처럼 쓰는 사고는 아주 흔한데, 0번지가 금지 구역이라서 그런 실수가 조용히 지나가지 않고 그 자리에서 요란하게 잡힌다.

임베디드의 세계는 다르다. 운영체제 없이 도는 작은 칩에서는 0번지가 멀쩡한, 심지어 중요한 메모리인 경우가 많다 — 여러 마이크로컨트롤러가 0번지 부근에 인터럽트 벡터 테이블(비상 연락망 같은 표)을 둔다. 그런 기계에서 0번지 읽기는 합법이고 의미가 있다.

문 0번지가 멀쩡한 메모리인 기계에서는, “없음” 표시와 “진짜 0번지”를 어떻게 구별하는가?

답 구별이 흐려지는 것이 사실이고, 그래서 임베디드 프로그래머는 이 경계를 의식하며 일한다. 더 흥미로운 것은 역사의 답이다 — 아예 0이 아닌 값을 “없음” 표시로 쓴 기계들이 실제로 있었다. 다음 사례가 그 이야기다.

● 널 포인터가 0이 아니었던 기계들

C 표준은 처음부터 신중했다. 소스 코드에 적은 상수 0이 널 포인터를 뜻한다는 것은 표준의 약속이지만, 그 널 포인터가 기계 안에서 **실제로 어떤 비트 패턴**으로 표현되는지는 기계의 자유로 남겨 두었다. 그리고 그 자유를 실제로 쓴 기계들이 있었다. Prime 50 시리즈는 세그먼트 07777, 오프셋 0이라는 값을 널로 썼고, CDC Cyber 180은 0xB00000000000이라는 특별한 패턴을, Lisp에 최적화된 Symbolics 머신은 “NIL 객체의 주소”라는 0 아닌 값을 널로 썼다. 이런 기계에서 널 포인터의 비트를 들여다보면 0이 하나도 아니다 — 그래도 소스 코드에서 포인터를 0과 비교하면 표준의 약속대로 참이 나온다. 소스의 0은 **기호**이고, 기계 속 표현은 **구현**이다 — 2장부터 이어 온 “기호와 표현의 분리”가 여기서도 반복된다.

오늘날의 주류 기계(인텔·AMD·ARM 등)는 전부 “모든 비트 0”을 널로 쓰므로 이 구별을 체감할 일은 드물다. 그러나 뒤에 볼 태그 포인터의 세계에서는 “없음을 나타내는 값이 전부 0은 아닌” 상황이 지금도 멀쩡히 살아 있다.

4.2 널 삼형제 — 이름만 닮은 세 가지

“널”이라 불리는 것이 C 주변에 셋 있다. 지금 한 번 구별해 두면 나중에 큰 혼란을 던다. 정식 취급은 제 7부(30장)에서 하고, 여기서는 얼굴만 익힌다.

- **널 포인터** — 방금 본 것. “아무 데도 가리키지 않는다”는 약속된 포인터 값이다. 포인터의 세계에 산다.
- **NULL** — C 소스 코드에서 널 포인터를 적을 때 쓰는 이름(매크로)이다. 최신 C(C23)에는 더 명확한 `nullptr` 라는 표기도 들어왔다. 역시 포인터의 세계에 산다.
- **NUL 문자** — 전혀 다른 물건이다. 문자표(7장)의 0번 자리에 있는, 값이 0인 문자 하나로, C에서는 `'\0'`이라 적는다. 크기 1바이트짜리 데이터이고, 문자열의 끝 표시로 쓰인다(33장). 문자의 세계에 산다.

셋 다 이름에 “널”이 들어가고 셋 다 어딘가에 0이 얹혀 있어서 뒤섞기 쉽지만, 포인터의 널과 문자의 NUL은 사는 세계가 다르다. “값이 우연히 다 0이라서 이름이 닮았을 뿐인 남남”이라 기억해 두면 된다.

4.3 정렬 — 두 칸짜리 짐은 아무 데나 못 넣는다

사물함 복도의 두 번째 구석 지식이다. 3장에서 기계는 여러 칸을 한 움큼에 집는다고 했다(워드). 그런데 기계의 손은 아무 위치에서나 움큼을 집도록 만들어져 있지 않다 — 대개 **자기 크기의 배수 번호에서만** 편하게 집는다. 4바이트짜리 수는 4의 배수 주소(100, 104, 108, ...)에, 8바이트짜리는 8의 배수 주소에 놓여 있을 때 한 번에 집힌다. 이 규칙이 **정렬(alignment)**이다.

78	56	34	12				
100	101	102	103	104	105	106	107

위처럼 100번(4의 배수)에서 시작하는 4바이트는 정렬이 맞다. 만약 같은 수를 102번에서 시작해 넣으면 — 정렬이 어긋난다. 그때 무슨 일이 일어나는지도 기계마다 다르다. 관대한 기계(인텔 계열)는 두 움큼으로 나눠 집느라 **느려질 뿐** 일은 해 준다. 엄격한 기계(옛 SPARC, 일부 ARM 시절)는 그 자리에서 오류(버스 폴트)를 내며 프로그램을 세웠다. “되는 기계에서만 되는 코드”라는 이식성 문제의 고전적 원천 중 하나다.

문 왜 기계는 아무 데서나 못 집는가? 사물함 비유로 무엇이 걸리는가?

답 기계의 손(버스)이 복도를 4칸짜리 격자로 보고 움직이기 때문이다. 100 103이 한 격자, 104 107이 다음 격자다. 102에서 시작하는 4바이트는 두 격자에 걸쳐 있어서, 한 번에 집으려면 격자 두 개를 모두 열고 필요한 조각을 이어 붙여야 한다 — 두 번 일하거나, 아예 거부하거나. 두 칸짜리 짐은 짝수 번호부터 넣으라는 사물함 관리 규칙과 같다.

4.4 하위 비트의 재주 — 태그 포인터

정렬 규칙에는 뜻밖의 부산물이 있다. 4의 배수인 수를 2진법으로 적으면 끝의 두 비트가 **항상 0**이고, 8의 배수라면 끝의 세 비트가 항상 0이다. 즉, 정렬된 주소의 하위 비트들은 언제나 0이라서 — **정보를 담는 공짜 공간**으로 쓸 수 있다.

이 재주를 **태그 포인터(tagged pointer)**라 부른다. 주소를 적어 두는 김에, 어차피 0인 끝자리에 작은 딱지(태그)를 끼워 넣는 것이다. 실제로 널리 쓰인다 — Lisp 계열 언어와 OCaml의 런타임은 “이 값은 정수인가, 객체 주소인가”를 하위 비트 태그로 구별하고, 자바스크립트 엔진과 가비지 컬렉터들도 같은

재주로 상태 표시를 포인터에 얹는다. 물론 공짜는 아니다 — 그 주소를 진짜로 쓰기 전에는 딱지를 떼야 (하위 비트를 도로 0으로) 한다.

여기서 앞의 널 이야기와 다시 만난다. 태그가 얹힌 세계에서는 “없음”을 나타내는 특별한 값조차 태그를 품고 있어서 **모든 비트가 0이 아닐 수 있다**. 예컨대 Lisp의 “없음”(NIL)은 실재하는 특별 객체의 주소다. C의 널 포인터가 주류 기계에서 전부 0으로 통일된 오늘날에도, 한 층 위의 런타임 세계에서는 “0 아닌 없음”이 여전히 현역이라는 이야기다.

문 프로그래머가 직접 태그 포인터 재주를 부려도 되는가?

답 C에서 기술적으로 가능하고 실제로 쓰이는 기법이지만, 함정이 많은 고급 기술이다 — 정렬 보장이 있어야 하고, 쓰기 전에 반드시 태그를 떼야 하고, 뒤에 배울 포인터 규칙(12장 프로버넌스, 31장)과도 얽힌다. 지금 단계의 결론은 하나면 된다: 주소의 하위 비트는 “그냥 수”가 아니라 정렬이 만들어 준 특별한 자리라는 것.

이 장의 세 가지 구석 지식 — 0번지의 특별함, 정렬, 하위 비트 — 은 모두 하나의 사실에서 나왔다. 주소는 수이지만, **아무 수나 똑같이 대접받는 것은 아니라는 것**. 복도에도 지리(地理)가 있다.

다음 장부터는 사물함에 담기는 내용물 쪽으로 눈을 돌린다. 표현의 사다리 첫 단은 정수다 — 음수를 비트로 담는 세 가지 방법이 경쟁했던 사연과 C23의 결단, 넘치는 수(오버플로), 그리고 비트를 통째로 밀고 당기는 시프트까지.

5 정수의 표현 — 부호, 오버플로, 시프트

이 장이 끝나면

표현의 사다리 첫 단이다. 부호 없는 정수가 어떻게 순환하는 수(모듈러) 인지, 음수를 비트로 담는 세 가지 방법이 어떻게 겨루다 2의 보수로 정리됐는지 — 그리고 C23이 마침내 무엇을 못박았는지 — 를 본다. 넘치는 수(오버플로)와, 비트를 통째로 밀고 당기는 시프트까지 챙기면, 정수에 관한 배경지식이 완성된다.

돌아보기 3장에서 여러 바이트를 이어 붙여 큰 수를 담는다 했고, 엔디안(넣는 순서)까지 보았다. 그런데 지금까지의 수는 전부 0 이상이였다. 음수는 — 비트 어디에 마이너스 기호를 담는가?

답: 담을 곳이 없다 — 비트에는 0과 1뿐, 마이너스 기호가 없다(2장). 그래서 음수 역시 **약속**으로 만든다. 어떤 비트 패턴을 음수로 “읽기로” 정하는 것이다. 그 약속을 정하는 방법이 하나가 아니었다는 것, 그리고 그 경쟁의 결말이 이 장의 한복판이다.

5.1 부호 없는 정수 — 시계처럼 도는 수

먼저 마이너스 걱정이 없는 세계부터 정리한다. n 비트를 그냥 2진수로 읽으면 0부터 $2^n - 1$ 까지, 2^n 개의 수가 담긴다(2장). 이것이 **부호 없는(unsigned)** 정수다. 8비트면 0 255다.

이 수의 세계에서 꼭 챙길 성질이 하나 있다. **끝이 처음과 이어져 있다**는 것이다. 255에서 1을 더하면 256이 아니라 — 256을 담을 아홉 번째 비트가 없으므로 — 0이 된다. 12시에서 한 시간이 지나면 1시가 되는 시계와 같은 구조다. 수학에서는 이런 셈을 **모듈러 산술**이라 부른다.

Σ 모듈러 산술 — 유한한 수의 정확한 수학

n 비트 부호 없는 정수의 덧셈·뺄셈·곱셈은 결과를 2^n 으로 나눈 나머지를 취하는 연산과 정확히 같다:

$$\text{결과} = (a + b) \bmod 2^n$$

8비트에서 $250 + 10 = 260 \bmod 256 = 4$ 다. 중요한 것은 이것이 “틀린 덧셈”이 아니라 **다른 덧셈** — 완전히 정의된, 예측 가능한 수학 — 이라는 점이다. C 표준도 부호 없는 정수의 넘침을 오류가 아니라 이 수학으로 정의한다.

이 순환에는 이름이 있다 — **오버플로(overflow)**, 정확히는 “감아 돌기”(wrap-around)다. 정의된 동작이라 해도, 예상하지 못한 자리에서 일어나면 사고가 된다.

● 256판의 벽 — 팩맨 킬 스크린

오락실 게임 팩맨에는 유명한 벽이 있다. 255판까지는 멀쩡히 진행되다가 256판째에 화면 오른쪽 절반이 뜻 모를 기호로 뒤덮이며 게임이 불가능해 진다. 판 번호를 **8비트**로 세던 코드가 255를 넘는 순간 0으로 감아 돌았고, “지금 몇 판인지”를 전제로 과일을 그리던 루틴이 엉뚱한 개수를 그리며 화면을 부순 것이다. 설계자가 “256판까지 갈 사람은 없다”고 여겼던 그릇의 크기가, 최고수들의 도전 앞에서 벽이 된 사례다 — 그릇의 크기는 언제나 누군가에게는 닿는 벽이 된다.

5.2 음수를 담는 세 가지 약속

이제 음수다. 관건은 n 비트 패턴의 절반쯤을 음수로 “읽기로” 약속하는 방식인데, 역사상 세 가지 약속이 실제로 쓰였다. 8비트로 -5 를 담는 경우로 비교한다.

첫째, 부호-크기(sign-magnitude). 사람의 표기를 그대로 흉내 낸다 — 맨 앞 비트를 마이너스 기호로 쓰고(1이면 음수), 나머지에 크기를 담는다. -5 는 $1_0000101$. 직관적이지만 두 가지 대가가 있다. $00000000(+0)$ 과 $10000000(-0)$, **0이 두 개 생긴다**. 그리고 덧셈 회로가 골치다 — 부호가 다른 두 수를 더하려면 “크기를 비교해서 큰 쪽에서 작은 쪽을 빼고 부호를 정하는” 별도 절차가 필요하다.

둘째, 1의 보수(ones' complement). 음수를 만들 때 모든 비트를 뒤집는다. 5 가 00000101 이니 -5 는 11111010 . 덧셈 회로가 한결 단순해지지만, 여전히 0이 두 개다(00000000 과 11111111) — 비교할 때마다 “두 0은 같은 것으로 친다”는 예외를 끌고 다녀야 한다.

셋째, 2의 보수(two's complement). 비트를 뒤집고 **1을 더한다**. -5 는 $11111010 + 1 = 11111011$. 언뜻 임의로 보이는 이 규칙이 사실은 가장 우아하다 — 0이 하나뿐이고, 무엇보다 **부호 없는 덧셈 회로를 그대로 쓰면 부호 있는 덧셈이 저절로 맞는다**. 별도 절차도 예외도 없다.

문 그런데 왜 이름이 “보수”인가? 그리고 1의 보수, 2의 보수라는 이름에서 1과 2는 무엇을 가리키는가?

답 **보수(補數, complement)**는 “채워서 어떤 기준을 완성해 주는 수”다. 십진법으로 먼저 감을 잡으면 쉽다 — 세 자리 수 304에 대해, 각 자리를 9까지 채워 주는 695를 **9의 보수**라 하고(모든 자리에서 $9 - \text{그 자리}$), 1000까지 채워 주는 696을 **10의 보수**라 한다($10^3 - 304$). 둘의 관계는 “9의 보수 + 1 = 10의 보수”다.

2진법에서 똑같은 일을 하면 이름의 뜻이 풀린다. 각 자리를 1까지 채워 주는 수 — 즉 11111111 에서 빼는 것 — 가 **1의 보수**인데, 2진법에서 $1 - \text{비트}$ 는 곧 비트 뒤집기라서 “뒤집는다”는 규칙이 나온 것이다. 그리고 2^n — 이진법의 “1000...0”, 즉 **2의 거듭제곱** — 에서 빼는 것이 **2의 보수**다. “뒤집고 1을 더한다”는 요령은 십진법의 “9의 보수 + 1 = 10의 보수”와 똑같은 관계다.

영어 표기도 여기서 정리해 둔다. 통용 표기는 **one's complement**와 **two's complement**다 (“1s”나 “2s” 같은 표기는 없다). 다만 전산학자 커누스(Knuth)는 재치 있는 구별을 주장했다 — 1의 보수는 “모든 자리의 1들에 대한 보수”이므로 복수 소유격 **ones' complement**가 옳고, 2의 보수는 “2”이라는 **하나의** 수에 대한 보수”이므로 단수 소유격 **two's complement**가 옳다는 것이다. 문법 트집 같지만 두 보수의 수학적 정의 차이(자리별 기준 vs 전체 기준)를 정확히 담은 구별이고 — 실제로 **C 표준 문서 자신이 이 구별을 채택했다**. 표준의 표현 조항은 세 방식을 “sign and magnitude”, “two's complement”, “**ones' complement**”라고 적는다. 커누스의 트집이 법전에 이긴 셈이다. 교과서 용어로는 2의 보수를 **기수 보수(radix complement)**, 1의 보수를 **감소 기수 보수(diminished radix complement)**라고도 부른다.

Σ 2의 보수가 우아한 이유 — 모듈러 산술의 재활용

2의 보수의 정체는 위의 모듈러 산술이다. $-x$ 를 $2^n - x$ 라는 패턴으로 담는 약속이므로, 8비트에 서 -5 는 $256 - 5 = 251$, 즉 11111011 이다. 그러면 $7 + (-5)$ 는 회로 입장에서 $7 + 251 = 258 \bmod 256 = 2$ — 답이 저절로 맞는다. 음수란 “모듈러 시계에서 반대 방향으로 세는 것”일 뿐이라서, 회로는 부호를 아예 몰라도 된다. 유일한 비대칭은 범위다 — 8비트에서 -128 부터 $+127$ 까지로, 음수가 하나 더 많다(-128 의 짝 $+128$ 이 없다).

5.3 세 약속의 경쟁, 그리고 C23의 결단

세 방식 모두 실제 기계에 쓰였다 — 부호-크기와 1의 보수는 초기 대형기들(UNIVAC, CDC 계열 등)에 실존했다. C가 표준화되던 1989년에도 그런 기계들이 현역이었기 때문에, C 표준은 어느 편도 들지 않았다. **세 가지 표현을 모두 허용한 것이다**. 이 중립은 공짜가 아니었다 — 표현이 다르면 같은 연산의 결과

비트가 달라지므로, 표준은 부호 있는 정수의 많은 동작을 “기계마다 다르다”로 남겨 둘 수밖에 없었다. 부호 있는 오버플로가 **정의되지 않은 동작**(41장)이 된 사연의 절반이 여기에 있다.

그사이 현실은 한쪽으로 수렴했다. 2의 보수의 회로 단순성이 압도적이라 수십 년간 새로 만들어진 CPU는 사실상 전부 2의 보수였고, 나머지 두 방식은 박물관으로 갔다. 그리고 **C23이 마침내 결단했다 — 부호 있는 정수의 표현은 2의 보수다**. 반세기의 관행이 표준의 약속으로 승격된 것이다(2장의 “바이트=8비트” 논의와 똑같은 무늬다).

문 표현이 2의 보수로 못박혔으니, 이제 부호 있는 오버플로도 “감아 돌기”로 정의된 것인가?

답 아니다 — 여기가 미묘하고 중요한 지점이다. C23이 못박은 것은 **표현** (음수가 어떤 비트 패턴인가)이지, **오버플로의 의미**가 아니다. 부호 있는 정수의 오버플로는 C23에서도 여전히 정의되지 않은 동작으로 남았다. 이유는 표현이 아니라 최적화다 — “부호 있는 수는 넘치지 않는다”는 전제가 컴파일러(11장)의 루프 분석과 재배열에 요긴해서, 표준은 그 전제를 유지하는 쪽을 택했다. 요약하면: 부호 없는 넘침 = 정의된 감아 돌기, 부호 있는 넘침 = 여전히 계약 밖. 실무 규칙은 23장에서 다룬다.

△ “오버플로가 나면 컴퓨터가 오류를 알려 준다”

그렇듯한 기대다 — 잘못된 일이 생기면 알려 주는 것이 도리이니까. 그러나 CPU의 덧셈 회로는 넘침의 순간 내부 신호(플래그)를 올릴 뿐, **프로그램을 세우거나 알리지 않는 것이 기본**이다. C도 마찬가지다 — 부호 없는 수는 조용히 감아 돌고, 부호 있는 수는 계약 밖(무슨 일이든 일어날 수 있음)이다. 팩맨의 화면이 요란하게 부서진 것은 오버플로 “경보”가 아니라 감아 둔 값이 낳은 **후속 사고**였다. 넘침의 감시는 기계가 아니라 프로그래머의 일이다 — 이것이 뒤에서 proven 같은 검증 도구가 산술을 검사하는 이유이기도 하다(34장, 43장).

5.4 시프트 — 비트를 통째로 밀기

정수의 비트를 다루는 기본 연산이 하나 더 있다 — **시프트(shift)**, 비트 열을 통째로 왼쪽이나 오른쪽으로 미는 것이다. 밀고 나면 두 가지 질문이 남는다. **밀려난 비트는 어디로 가고, 빈자리는 무엇으로 채우는가**.

왼쪽 시프트는 답이 하나다. 위로 밀려난 비트는 버려지고, 아래 빈자리는 0으로 채운다. 00010110을 왼쪽으로 한 칸 밀면 00101100 — 십진법에서 끝에 0을 붙이면 10배가 되듯, 2진법의 왼쪽 한 칸은 **2배**다.

오른쪽 시프트는 답이 둘이다. 빈자리(맨 위)를 무엇으로 채우는가에 따라 갈린다.

- **논리 시프트(logical)**: 0으로 채운다. 부호 없는 수에 맞는 방식이고, 오른쪽 한 칸은 2로 나눈 몫이 된다.
- **산술 시프트(arithmetic)**: **부호 비트를 복사해** 채운다. 2의 보수 음수는 맨 위 비트들이 1로 차 있으므로(위의 $-5 = 11111011$), 1을 채워야 “2로 나누기”라는 뜻이 유지된다. 0을 채워 버리면 음수가 갑자기 거대한 양수처럼 읽힌다.

그래서 CPU들은 오른쪽 시프트 명령을 두 벌(논리/산술) 갖고 있고, C에서는 부호 없는 수의 오른쪽 시프트는 논리로, 부호 있는 음수의 오른쪽 시프트는 — 오랫동안 “기계마다 다르다”였다가 사실상 모든 구현이 산술을 쓰는 관행으로 수렴했다. 2의 보수 확정과 나란히, 관행을 약속으로 승격하는 방향의 정리다.

문 비트를 밀고 채우는 규칙까지 알아야 할 만큼, 시프트가 중요한 연산인가?

답 중요하다 — 두 가지 이유에서다.

첫째, **가장 싼 연산이기 때문이다**. 시프트는 회로 입장에서 배선을 옆으로 옮기는 수준의 일이라, 거의 모든 CPU에서 한 박자짜리 최속 연산에 속한다. 방금 본 대로 왼쪽 k 칸은 2^k 배, 오른쪽 k 칸은 2^k 으로 나눈 몫이므로 — 2의 거듭제곱 곱셈·나눗셈을 시프트로 바꾸는 것은 고전적인 속도 요령이었다. 다만 오늘의 C에서는 그 요령을 **사람이 부릴 필요가 없다**. 소스에는 뜻 그대로 $x * 2$, $x / 8$ 이라 쓰면 되고, 컴파일러(11장의 편집자)가 알아서 시프트로 바꿔 준다. 가독성을 내주고 얻을 것이 없는 자리다.

둘째, **비트의 세계를 다루는 기본 동작이기 때문이다**. 여러 값을 한 정수의 비트 자리들에 나눠 담고 꺼내는 일 — 7장에서 본 UTF-8 바이트 조립, 4장의 태그 포인터, 색상값(RGB)의 분해, 하드웨어 레지스터의 플래그 읽기 — 이 전부가 “원하는 자리로 밀고, 필요한 비트만 남기는” 시프트+마스크의 조합이다. 수의 곱셈으로서의 시프트는 컴파일러에게 넘어갔지만, **자리 배치 도구**로서의 시프트는 시스템 프로그래머의 일상 언어로 남아 있다. C 문법에서의 실제 사용은 24장에서 다룬다.

문 8비트짜리 수를 여덟 칸, 혹은 그 이상 밀면 어떻게 되는가? 상식적으로는 전부 밀려나 0이 될 것 같은데.

답 바로 그 “상식”이 기계마다 달랐다는 것이 함정이다. 시프트 칸수는 CPU 안에서 몇 비트짜리 회로가 처리하는데, 꼭 이상의 칸수가 들어왔을 때의 대응이 갈렸다 — 어떤 CPU 계열(인텔 x86)은 칸수의 아래 몇 비트만 보고 나머지를 무시해서 32비트 수를 32칸 밀면 **그대로**이고, 다른 계열(옛 ARM 등)은 정말로 다 밀어서 **0**이 된다. 같은 코드가 기계마다 다른 답을 내는 것이다. C 표준의 대응은 이제 익숙한 패턴이다 — 어느 편도 들 수 없으니, **꼭 이상의 시프트는 정의되지 않은 동작**으로 계약 밖에 두었다. “기계들이 서로 다르게 대응하는 곳은 표준이 약속을 포기한다” — 이 무늬는 41장에서 정식으로 다시 만난다.

5.5 부호 확장 — 좁은 그릇에서 넓은 그릇으로

시프트의 “무엇으로 채우는가”라는 질문은 한 군데서 더 나타난다. 8비트 그릇에 든 수를 16비트 그릇으로 옮겨 담을 때다. 늘어난 위쪽 여덟 칸을 무엇으로 채울 것인가.

부호 없는 수는 답이 자명하다 — **0으로 채운다**(zero extension). 8비트의 11111011(=251)은 16비트의 00000000 11111011(=251)이 된다. 값이 그대로다.

부호 있는 수에서는 같은 방법이 사고를 낸다. 8비트의 11111011은 2의 보수로 -5인데, 위를 0으로 채우면 16비트의 00000000 11111011 — 맨 앞 비트가 0이니 **양수 251**로 읽힌다. -5가 그릇을 옮기다가 251이 된 것이다. 올바른 답은 산술 시프트와 같은 요령이다 — **부호 비트를 복사해 채운다**. 11111111 11111011, 여전히 -5다. 이것이 **부호 확장**(sign extension)이다.

문 1을 잔뜩 채웠는데 왜 값이 그대로인가? 우연인가?

답 우연이 아니라 모듈러 수학의 필연이다. 2의 보수에서 8비트의 -5는 $2^8 - 5 = 251$ 이라는 패턴이었고, 16비트의 -5는 $2^{16} - 5 = 65531$ 이라는 패턴이다. 그런데 $65531 = 251 + 255 \cdot 256$ — 이진법으로 적으면 정확히 “원래 패턴 위에 1을 여덟 개 얹은 것”이다. 부호 비트 복사를 “ 2^8 에 대한 보수를 2^{16} 에 대한 보수로 갈아 끼우는” 산술을 비트 복사 한 번으로 해치우는 요령이다. 2의 보수의 우아함이 여기서도 일한다 — 부호-크기나 1의 보수였다면 이런 공짜 확장은 없다.

거꾸로 넓은 그릇에서 좁은 그릇으로 **줄여** 담으면 위쪽 비트들이 그냥 잘려 나간다 — 값이 그릇에 안 들어가면 소리 없이 망가진다는 점에서 오버플로의 사촌이다. C는 계산 전에 작은 정수를 자동으로 넓히는

규칙 (정수 승격)을 갖고 있고, 넓히기·줄이기가 언제 일어나며 무엇이 위험한지는 C의 정수 타입과 함께 23·24장에서 정식으로 다룬다 — 이 장의 그림 (0 채움 / 부호 복사 / 잘림)이 그때의 밑천이다.

정수의 배경지식이 완성됐다. 부호 없는 수는 시계처럼 도는 모듈러의 세계이고, 음수는 세 가지 약속의 경쟁 끝에 2의 보수로 정리되어 C23이 못박았으며, 넘침은 조용하고, 시프트와 그릇 옮기기(확장)는 채움의 방식 까지 알아야 뜻이 선다. C의 정수 **타입들**과 실무 규칙은 23·24장에서 이 배경 위에 세운다.

다음 장은 사다리의 다음 단이다 — 정수 너머, 소수점 있는 수를 담는 두 가지 방법과 IEEE 754라는 계약을 만난다.

6 수의 표현 ↔ IEEE 754라는 계약

이 장이 끝나면

표현의 사다리 첫 단이다. 소수점 있는 수를 사물함에 담는 두 가지 방법 — 고정소수점과 부동소수점 — 을 구별하게 되고, 부동소수점의 난립을 통일한 IEEE 754라는 계약을 만난다. 컴퓨터의 수가 유한하고 근사적이라는, 이 책에서 가장 중요한 사실 하나가 여기서 자리 잡는다.

돌아보기 5장에서 부호 있는 정수까지 비트에 담았다 — 음수를 담는 세 가지 방법이 겨루다 2의 보수로 정리되는 것까지 보았다. 그런데 여전히 전부 정수다. 1.5 같은 수는 — 비트로 어떻게 담는가?

답 비트 자체에는 소수점이 없다. 그래서 소수점은 **약속**으로 만든다 — 음수를 “약속”(2의 보수)으로 만들었던 것과 같은 이치다. “소수점이 어디에 있다고 치고 읽을 것인가”를 정하는 방식이 두 갈래로 갈리는데, 소수점을 못박아 두는 쪽(고정)과 소수점을 데이터에 실어 움직이는 쪽(부동)이다. 이 장이 그 두 갈래의 이야기다.

6.1 고정소수점 — 소수점을 약속으로 못박다

첫 번째 방법은 놀랄 만큼 단순하다. 그냥 정수를 쓰되, 단위를 바꾼다.

돈 계산이 좋은 예다. 1,500.25원을 저장하고 싶으면, “우리 장부는 전부 전(錢, 0.01원) 단위로 적는다”고 약속하고 정수 150025를 저장하면 된다. 소수점은 어디에도 저장되지 않는다 — 읽고 쓰는 모든 사람이 “끝에서 두 자리 앞에 소수점이 있다고 친다”는 약속을 공유할 뿐이다. 이것이 **고정소수점(fixed point)**이다. 소수점의 위치가 약속으로 고정되어 있다.

고정소수점의 미덕은 정수 그 자체라는 것이다. 더하기와 빼기가 정수 연산만큼 빠르고 정확하며, 어렵지 않다. 그래서 금액 계산과, 소수점 하드웨어가 없는 작은 임베디드 칩에서 지금도 현역이다. 약점은 뺏함이다 — 아주 큰 수와 아주 작은 수를 한 약속으로 감당할 수 없다. 전 단위 장부에 은하의 질량과 원자의 질량을 같이 적을 수는 없는 노릇이다.

6.2 부동소수점 — 소수점을 데이터에 싣다

두 번째 방법은 과학 시간에 배운 **과학적 표기법**을 기계에 옮긴 것이다. 6.02×10^{23} 처럼 수를 “알맹이 숫자”와 “10을 몇 번 곱했는가”로 나눠 적는 방식 말이다. 알맹이(가수)와 곱한 횟수(지수)를 둘 다 데이터로 저장하면, 소수점이 지수를 따라 **떠다닌다** — 그래서 **부동소수점(floating point)**이다. 기계는 10 대신 2를 쓴다: 수를 가수 $\times 2^{\text{지수}}$ 꼴로 담는다.

이 방식의 힘은 **같은 비트 수로 어마어마한 범위를 감당한다**는 것이다. 은하도 원자도 한 형식에 담긴다. 대가는 정밀도다 — 알맹이 숫자의 자리 수가 유한하므로, 그 자리 수를 넘는 부분은 **반올림으로 버려진다**.

△ “컴퓨터는 수학을 잘하니까, 소수 계산은 당연히 정확하다”

그렇듯하다 — 계산기가 틀리는 것을 본 적이 없으니까. 그러나 부동소수점이 담을 수 있는 수는 **유한 개**뿐이고, 담지 못하는 수는 가장 가까운 이웃으로 반올림된다. 놀랍게도 0.1조차 그렇다 — 0.1은 2진법으로 무한소수라서 (아래 수학 박스), 기계 속의 “0.1”은 사실 0.1에 아주 가까운 다른 수다. 작은 어긋남은 계산을 거치며 쌓일 수 있다. 이것은 컴퓨터의 결함이 아니라 유한한 표현의 필연이고, 잘 다루는 법이 따로 있다(39장). 지금 기억할 것은 하나다: **부동소수점은 정확한 수가 아니라 성실한 근사다**.

Σ 왜 0.1은 2진법으로 못 떨어지는가

유한 소수로 떨어지는 분수는 분모가 기수의 거듭제곱 인수로만 이루어진 경우뿐이다. 십진법(기수 $10 = 2 \times 5$)에서 $\frac{1}{10}$ 은 한 자리로 끝난다. 그러나 2진법의 기수는 2뿐이라서, 분모에 5가 들어 있는 $\frac{1}{10}$ 은 절대 유한하게 끝나지 않는다:

$$0.1_{10} = 0.0001100110011..._2$$

$\frac{1}{3}$ 이 십진법에서 0.333...으로 무한한 것과 같은 이치다. 유한한 가수는 이 무한을 어딘가에서 잘라야 하고, 그 잘림이 근사의 근원이다.

6.3 근사가 일으키는 세 가지 사건

“성실한 근사”가 실전에서 어떤 얼굴로 나타나는지, 수치로 미리 구경해 둔다. (여기서는 지면 계산으로 보이고, C 코드로 직접 실행해 보이는 것은 39장이다.)

사건 1 — $0.1 + 0.2 \neq 0.3$. 프로그래밍 세계에서 가장 유명한 등식 아닌 등식이다. 방금 본 대로 0.1도 0.2도 0.3도 2진법으로는 무한소수라서, 기계에는 각각 “가장 가까운 이웃”이 대신 담긴다. 그런데 0.1의 이웃과 0.2의 이웃을 더한 결과가, 하필 0.3의 이웃과 **다른 쪽**으로 반올림된다. 64비트 형식으로 실제 값을 적으면 —

- 기계 속 $0.1 + 0.2 = 0.30000000000000004440...$
- 기계 속 $0.3 = 0.29999999999999998889...$

둘 다 0.3에 성실하게 가깝지만, **서로 같지는 않다**. 그래서 “같은가?”라고 물으면(==) 기계는 정직하게 “아니오”라고 답한다.

내부 표현을 직접 들여다보면 사건의 전모가 한눈에 보인다. 64비트 형식은 [부호 1비트 | 지수 11비트 | 가수 52비트]로 나뉘는데, 이 네 수의 64비트를 16진법으로 적으면 —

수	64비트 내부 표현 (16진)
0.1	3FB9 9999 9999 999A
0.2	3FC9 9999 9999 999A
0.3	3FD3 3333 3333 3333
$0.1 + 0.2$	3FD3 3333 3333 3334

읽을거리가 세 겹이다. 첫째, 0.1과 0.2의 가수에 늘어선 9999...는 2진 무한소수 0011 의 순환이 16진법으로 보인 모습이고, **끝자리가 9가 아니라 A인 것은** 무한을 52비트에서 자르며 반올림해 올린 흔적이다 — 수학 박스에서 본 “잘림”이 비트에 그대로 찍혀 있다. 둘째, 0.3의 가수 3333...도 같은 순환의 다른 위상이며, 이쪽은 마지막에서 내림이 됐다. 셋째 — 핵심이다 — 0.3과 $0.1+0.2$ 는 **마지막 비트 하나만** 다르다. 눈금 하나(전문 용어로 1 ulp) 차이의 바로 옆 이웃인 것이다. ==는 이 한 비트의 차이도 다름으로 답하므로, 부동소수점의 “같음”은 눈금 하나의 어긋남에도 깨진다.

사건 2 — 그러면 비교는 어떻게 하는가. 부동소수점의 세계에서 “같다”는 질문 자체를 바꿔야 한다 — “정확히 같은가”가 아니라 “충분히 가까운가”로. 허용 오차(전통적으로 **엡실론**, ϵ 이라 부른다)를 하나 정하고, 두 수의 차이가 그 안에 들면 같다고 치는 것이다:

$$|a - b| < \epsilon \quad (\text{예: } \epsilon = 10^{-9})$$

이것이 기본형이고, 실전에는 한 겹이 더 있다 — 수가 커지면 이웃 사이 간격도 커지므로(아래 사건 3), 고정된 ϵ 대신 **수의 크기에 비례하는** 허용치(상대 오차)를 쓰는 것이 안전하다. 두 방식의 구체적인 사용

법과 함정은 39장에서 C 코드로 다룬다. 지금 기억할 것은 한 문장이다: **부동소수점을 ==로 비교하는 코드는 거의 언제나 의심 대상이다.**

사건 3 — 큰 수 곁에서 작은 수는 사라진다. 유효숫자가 유한하다는 것은, 크기가 너무 다른 두 수를 한 그릇에서 만나게 하면 작은 쪽이 **통째로 사라진다**는 뜻이기도 하다. 32비트 float(유효 십진 약 7자리) 로 시연하면 —

- 8.000000001을 담는 순간: 유효 7자리를 넘는 꼬리가 잘려 그냥 8.0이 된다. **더하기도 전에, 저장에서 이미 사라졌다.** 내부 표현으로 보면 더 선명하다 — float의 8.0은 4100 0000인데, 8.000000001을 담아도 가장 가까운 눈금이 8.0이라서 **똑같은 4100 0000**이 된다. 십진 표기로는 다른 두 수가, 비트로는 완전히 같은 하나의 수다.
- $8.0 + 0.000000008$: 결과는 그대로 8.0, 비트도 그대로 4100 0000 이다. 왜인가 — 8.0 크기에서 float의 눈금 간격(이웃 사이 거리)을 계산하면 $2^3 \times 2^{-23} = 2^{-20} \approx 0.000000095$ 다. 더한 0.000000008은 눈금 반 칸에도 못 미쳐서, 반올림하면 제자리다. 이것을 **흡수**라 부른다.
- 반대로 $8.0000001 - 8.0$ 처럼 **거의 같은 두 수의 뺄셈**은 앞자리들이 전부 지워지고 끝자리의 어림만 남는다 — 유효숫자 7자리로 시작한 계산이 유효숫자 한두 자리짜리 답을 내는 것이다. 이쪽은 **상쇄(cancellation)**라 부르며, 수치 계산에서 정밀도를 갉아먹는 주범이다.

세 사건의 뿌리는 하나다 — 표현 가능한 수들이 **띄엄띄엄한 눈금**이고, 눈금 간격은 수가 클수록 넓어진다. 0 근처에서는 눈금이 촘촘해 10^{-300} 같은 수도 구별하지만, 10^{16} 쯤 가면 눈금 간격이 1을 넘어 **정수조차 건너뛰다**. “몇 자리까지 믿을 수 있는가”(유효숫자)라는 감각이 부동소수점 사용의 전부라 해도 과언이 아니다.

6.4 난립, 그리고 IEEE 754라는 계약

부동소수점이라는 착상은 일찍부터 있었지만, **어떻게** 담을 것인가 — 가수와 지수에 몇 비트씩 줄 것인가, 반올림은 어느 쪽으로 할 것인가 — 는 오랫동안 회사마다 달랐다. IBM 대형기, DEC의 VAX, Cray 슈퍼컴퓨터가 저마다 다른 형식을 썼다. 같은 프로그램이 기계를 옮기면 **다른 답**을 내놓았고, 수치 계산 프로그래머들은 기계마다 계산의 버릇을 새로 익혀야 했다.

1985년, 이 난립을 IEEE 754라는 표준이 통일했다. 형식(부호 1비트 + 지수 + 가수), 반올림 규칙, 무한대와 “수 아님”(NaN) 같은 특수값의 처리까지 못박은 정밀한 계약이다. 오늘날 사실상 모든 CPU와 GPU가 이 계약을 따른다 — 32비트 형식(C의 float)과 64비트 형식(C의 double)이 그 대표다.

시대를 눈여겨보면 재미있다. 부동소수점의 계약(IEEE 754, 1985)과 C 언어의 계약(ANSI C, 1989)은 같은 몇 해 사이에 태어났다. 벤더마다 제멋대로이던 것을 건디다 못해 산업 전체가 “계약서를 쓰자”로 돌아선 시대였다 — 이 “계약의 시대” 이야기는 10장에서 다시 만난다.

문 32비트 float와 64비트 double은 실무에서 어떻게 갈라 쓰는가?

답 기본값은 double이다. 64비트 형식은 십진 유효숫자로 약 15 16자리를 감당해서 대부분의 계산에 여유가 있다. float(약 7자리)는 메모리와 대역폭이 아쉬운 곳 — 대량의 좌표, 그래픽, 기계학습 — 에서 크기의 이득으로 선택한다. “정밀도가 필요해서 double, 물량이 필요해서 float” 라고 기억하면 대강 맞다. C에서의 실제 사용은 39장에서 다룬다.

문 5장의 정수 오버플로와 이 장의 반올림은 둘 다 “유한한 그릇” 문제인데, 성격이 같은 문제인가?

답 뿌리는 같고 증상이 다르다. 정수의 그릇은 **범위**가 유한해서 끝에서 넘치고(오버플로 — 넘치기 전까지는 완전히 정확하다), 부동소수점의 그릇은 **정밀도**가 유한해서 어디서나 조금씩 어림된다(대

신 범위는 어마어마하다). 정확하지만 좁은 그릇과, 넓지만 어림하는 그릇 — 어느 그릇을 쓸지 고르는 감각이 수를 다루는 기본기이고, C의 타입 선택 (23장, 39장)이 바로 그 고르기다.

표현의 사다리 첫 단을 정리하면 이렇다. 소수점은 비트에 없고 약속에 있다. 약속을 못박으면 고정소수점, 데이터에 실으면 부동소수점이고, 부동소수점의 약속은 IEEE 754라는 계약으로 통일되어 있다. 그리고 그 계약 아래의 수는 정확한 수가 아니라 성실한 근사다.

다음 단은 글자다. 사물함에 “안녕”을 담으려면 무엇을 약속해야 하는가 — 문자와 텍스트의 세계, 그리고 그 약속들이 어긋나며 남긴 흉터들의 이야기다.

7 문자와 텍스트 ↔ 표준 속의 흉터

이 장이 끝나면

사물함에 글자를 담는 법을 배운다 — 문자는 결국 번호라는 것, 그 번호표 (문자 코드)가 어떤 역사를 거쳐 유니코드와 UTF-8에 이르렀는지, 그리고 그 역사가 C 문법에 남긴 흉터(삼중자)까지. 한국어를 쓰는 독자에게는 유난히 실감 나는 장이다.

돌아보기 6장에서 소수점은 비트에 없고 약속에 있다고 했다. 그러면 글자는 — 비트에 “가”라는 모양이 들어 있는 것인가?

답 아니다, 같은 이치다. 비트에는 모양도 소리도 없다. 있는 것은 수뿐이고, “어떤 수가 어떤 글자인가”라는 **번호표의 약속**이 따로 있다. 글자를 담는다는 것은 번호를 담는 것이고, 텍스트란 번호의 열이다. 이 장은 그 번호표들의 이야기다.

7.1 문자는 번호다

기계에 “A”를 저장하는 방법은 하나뿐이다 — “A는 65번으로 한다”는 약속을 만들고, 65라는 수를 저장하는 것이다. 이런 약속의 표, 즉 글자와 번호의 대응표를 **문자 집합**(character set) 또는 문자 코드라 부른다.

문제는 이 표가 **여러 개**였다는 것이다. 회사마다, 나라마다 자기 표를 만들었다. 같은 번호가 표마다 다른 글자를 가리켰고, 표가 다른 두 기계가 텍스트를 주고받으면 글자가 엉켰다. 이 장의 나머지는 그 엉킴의 역사이고, 역사의 각 굽이가 오늘의 C와 컴퓨팅에 흔적을 남겼다.

7.2 ASCII와 EBCDIC — 두 개의 표

1960년대에 두 갈래의 표가 자리 잡았다. IBM의 대형 컴퓨터 세계는 **EBCDIC**이라는 표를 썼다 — 천공 카드 시절의 코드에서 자라난 표라서 알파벳이 연속 번호가 아닌 등 지금 눈에는 기묘한 배치였지만, IBM의 세계에서는 그것이 법이었다.

다른 한쪽에서 통신 업계를 중심으로 만들어진 표가 **ASCII**다. 7비트, 즉 128칸짜리 작은 표에 영어 대소문자·숫자·문장부호, 그리고 화면에 안 보이는 **제어 문자**들을 담았다. 제어 문자는 “종이를 한 줄 올려라”, “경고음을 울려라”처럼 글자가 아니라 장치에 내리는 지시다 — 이들의 사연은 다음 장(스트림)에서 만난다. 그리고 이 표의 **0번 자리**에 앉은 것이 4장에서 얼굴을 익힌 **NUL 문자**다. “아무 글자도 아님”을 뜻하는 문자로, 나중에 C가 문자열의 끝 표시로 채용한다(33장).

ASCII가 이긴 표가 되었다 — 오늘날 거의 모든 문자 코드가 ASCII를 안쪽에 품고 있다. 그러나 128칸은 영어만으로도 빠듯했고, 세계의 글자를 담기에는 어림도 없었다. 여기서 흉터의 역사가 시작된다.

7.3 ISO 646 — 국가 변형과 C의 삼중자

첫 번째 미봉책은 ASCII의 국제판인 **ISO 646**이었다. 아이디어는 이랬다: “표는 하나로 쓰되, 나라마다 덜 중요한 몇 칸을 자기 글자로 바꿔 쓰게 허용한다.” 그렇게 바꿔 쓰도록 지정된 칸에 하필 [] { } \ | # 같은 기호들이 있었다.

결과는 프로그래머에게 재앙이었다. 덴마크의 단말기에서 C 코드를 열면 중괄호 자리에 덴마크 글자가 보였다. 같은 번호인데 표가 달라 다른 글자가 찍힌 것이다.

● ₩ 표시 — 한국에 남은 ISO 646의 흉터

한국도 이 역사의 당사자다. 한국판 표(KS X 1003)는 백슬래시 \ 자리를 원화 기호 ₩로 바꿨다. 그 시절 한국 컴퓨터에서 파일 경로는 C:\WProgram Files\W...로 보였고 — 놀랍게도 이 흉터는 지금도 만질 수 있다. 오늘날의 한국어 윈도우에서도 일부 글꼴은 백슬래시 번호를 ₩로 그린다. 같은 번호, 같은 비트인데 표(글꼴)가 ₩를 그리는 것이다. “기억된 것과 그것을 읽는 방법은 별개”라는 이 책의 후렴구를, 한국 독자는 화면에서 매일 목격해 온 셈이다.

C는 이 혼란의 한복판에서 표준화됐다(10장). 중괄호가 안 보이는 나라에서도 C를 쓸 수 있어야 했으므로, C89는 기괴한 우회로를 마련했다 — **삼중자(trigraph)**다. ??<라고 적으면 {로, ??/라고 적으면 \로 읽어 주는 문자 세 개짜리 암호였다. 언어 문법에 새겨진, 문자 코드 전쟁의 흉터다.

문 삼중자는 지금도 쓰이는가?

답 아니다 — 그리고 그 최후가 근래 C 역사의 상징적 장면이다. 유니코드 시대가 오며 삼중자는 존재 이유를 잃었고, 오히려 ??! 같은 문자열이 뜻밖에 변환되는 함정으로만 남았다. C23이 삼중자를 표준에서 **제거**했다. 1989년에 새겨진 흉터가 2023년에야 지워진 것이다 — 표준이 무언가를 들이는 일보다 **빼는** 일이 얼마나 느린지 보여 주는 사례이기도 하다.

7.4 8비트의 백가쟁명, 그리고 한글

바이트가 8비트로 자리 잡자(2장) 표를 256칸으로 늘린 **ISO 8859** 계열이 나왔다 — 앞 128칸은 ASCII 그대로, 뒤 128칸에 각 언어권 글자를 담는 방식이다(서유럽용 Latin-1 등). 그러나 뒤 128칸을 언어권마다 다르게 쓰는 구조라, 표를 잘못 짚으면 글자가 엉키는 문제는 그대로였다.

그리고 256칸으로는 어림도 없는 글자들이 있었다. 한글, 한자, 가나 — 동아시아의 글자는 수천, 수만이다. 해법은 **한 글자에 바이트 여러 개**를 쓰는 멀티바이트 인코딩이었다(한국의 EUC-KR 등). 글자마다 길이가 다른 데다 나라마다 방식이 달라, 표를 잘못 짚은 한글 문서가 뜻 모를 기호의 행렬로 깨지는 일 — 이른바 “글자 깨짐” — 은 이 시대 한국 컴퓨터 사용자의 일상이었다.

7.5 유니코드와 UTF-8 — 하나의 표, 영리한 답기

근본 해법은 결국 하나뿐이었다. **세상 모든 글자를 하나의 표에 넣자**. 그것이 1990년대의 **유니코드**다. 글자마다 고유 번호(코드 포인트)를 주고 U+AC00(가)처럼 적는다. 표의 크기는 백만 칸이 넘는다.

남은 문제는 그 큰 번호를 바이트에 **어떻게 담느냐**였다. 모든 글자를 4바이트씩 쓰면 낭비가 크고, 무엇보다 산더미 같은 기존 ASCII 텍스트와 호환되지 않는다. 답이 된 것이 **UTF-8**이라는 가변 길이 인코딩이다 — ASCII 범위의 글자는 1바이트 그대로, 그 밖의 글자는 2~4바이트로 담는다. 기존 ASCII 파일은 그 자체로 올바른 UTF-8이라는 절묘한 설계 덕에, UTF-8은 오늘날 웹과 소스 코드의 사실상 유일한 표준이 됐다. 이 책의 예제와 C23의 `u8` 문자열이 쓰는 것도 UTF-8이다(33장).

Σ UTF-8의 바이트 구조

코드 포인트의 크기에 따라 길이가 달라진다. 첫 바이트의 앞 비트들이 “이 글자가 몇 바이트짜리 인지”를 알리고, 이어지는 바이트는 전부 10으로 시작한다:

범위	바이트 수	비트 배치
U+0000 U+007F	1	0xxxxxxx (= ASCII)
U+0080 U+07FF	2	110xxxxx 10xxxxxx
U+0800 U+FFFF	3	1110xxxx 10xxxxxx 10xxxxxx

U+10000 U+10FFFF	4	11110xxx 10xxxxxx x3
------------------	---	----------------------

이어지는 바이트가 항상 10으로 시작하므로, 텍스트 중간 아무 데서 깨어나도 글자의 경계를 되찾을 수 있다(자기 동기화). 한글 음절(U+AC00 부근)은 3바이트 구간에 산다.

⚠ “한 글자는 1바이트다”

ASCII 시대의 직관이 남긴 오개념이고, C를 배울 때 특히 위험하다 — C의 char 타입은 이름과 달리 “글자”가 아니라 **바이트**이기 때문이다(33장). UTF-8에서 영문 A는 1바이트지만 한글 가는 3바이트다. “안녕”이라는 두 글자는 여섯 바이트다. 글자 수와 바이트 수는 다른 물건이고, 이 구별이 무너진 코드는 한글 처리에서 반드시 사고를 낸다.

문 이 어지러운 역사를 입문자가 왜 지금 알아야 하는가? UTF-8만 쓰면 되는 것 아닌가?

답 새로 만드는 것은 UTF-8만 쓰면 된다 — 그것이 오늘의 관행이고 이 책의 전제다. 그러나 두 가지 이유로 역사가 필요하다. 첫째, 세상의 파일과 시스템에는 옛 인코딩이 여전히 흐르고 있어서, 글자 깨짐을 만났을 때 “표를 잘못 짚었구나”를 진단할 수 있어야 한다. 둘째, C 자신이 이 역사의 산물이라 — char라는 이름, NUL 종단, 삼중자의 잔해 — 역사를 알면 C의 기묘한 구석들이 전부 사연 있는 상처로 읽힌다. 외울 것은 없다. “문자는 번호이고, 표가 여러 개였다가 유니코드로 통일됐고, 답는 방식은 UTF-8이 이겼다” — 이 세 문장이면 충분하다.

다음 장은 표현의 사다리 마지막 단이다. 글자를 담을 줄 알게 됐으니, 이제 글자가 **흐르**는 이야기다 — 컴퓨터의 입출력이 왜 “문자가 한 줄씩 흘러가는 것”으로 생겼는지, 그 답이 종이 카드와 타자기의 시대에 있다.

8 스트림의 기원 ↔ 펀치카드, 라인프린터, 프린터 터미널

이 장이 끝나면

컴퓨터의 입출력이 왜 “문자가 한 줄씩 흐르는 것”의 모습인지 알게 된다. 답은 종이의 시대에 있다 — 펀치카드, 라인프린터, 그리고 타자기를 닮은 프린터 터미널. `\n`이라는 두 글자와 `tty`라는 이상한 이름의 유래도 여기서 풀린다. 이 장이 끝나면 곧 만날 첫 프로그램의 `printf`가 어디에 대고 말하는 것인지 미리 이해하게 된다.

돌아보기 7장에서 제어 문자 — “종이를 한 줄 올려라” 같은, 장치에 내리는 지시 — 가 문자표의 한 구석에 산다고 했다. 글자들의 표에 왜 장치 조작 지시가 끼어 있는가?

답 그 표가 만들어진 시대에는 텍스트가 곧 **장치의 움직임**이었기 때문이다. 글자는 화면에 그려지는 것이 아니라 종이에 물리적으로 찍히는 것이었고, “다음 줄로”는 실제로 기계 부품이 움직이는 동작이었다. 그 시대의 장치들을 구경하면, 오늘의 입출력이 왜 이런 모양인지 전부 풀린다.

8.1 펀치카드 — 한 장이 한 줄이다

초기 컴퓨팅의 입력은 **펀치카드**였다. 뾰뚱한 종이 카드에 구멍을 뚫어 글자를 기록한다 — 구멍의 조합이 글자 하나이고, 카드 한 장에 글자가 **80칸** 들어간다. 프로그램 한 줄을 카드 한 장에 뚫고, 프로그램 전체는 카드 뭉치가 된다. 뭉치를 기계에 먹이면 카드가 한 장씩 빨려 들어가며 읽힌다.

여기서 두 가지 유산이 태어났다. 첫째, “**한 장 = 한 줄**”이라는 감각. 텍스트는 줄들의 열이라는, 지금은 공기처럼 당연한 관념이 물리적 실물에서 왔다. 둘째, **80이라는 수**. 카드의 80칸은 이후 터미널 화면의 80컬럼으로, “코드 한 줄은 80자 안쪽으로”라는 오늘날의 코딩 관행으로 반세기를 건너 살아남았다.

문 카드 뭉치를 떨어뜨리면 어떻게 되는가?

답 대참사다 — 순서가 섞인 카드 수백 장이 곧 프로그램이 뒤섞인 것이다. 그래서 카드 구석에 일련번호를 뚫어 두고 기계로 다시 정렬하는 요령이 있었다. 카드 한 귀퉁이를 사선으로 잘라 뒤집힌 카드를 눈으로 찾게 한 것도 같은 지혜다. 우스개 같지만, “데이터에 순서 번호를 붙여 두면 섞여도 복구할 수 있다”는 착상은 지금도 통신과 데이터베이스 설계의 기본기다.

8.2 라인프린터 — 줄 단위로 찍는 출력

출력 쪽의 실물은 **라인프린터**였다. 이름 그대로 **한 번에 한 줄**을 통째로 종이에 찍는 인쇄기다. 계산 결과는 화면이 아니라 연속 용지에 줄줄이 찍혀 나왔다. 입력은 카드 한 장씩(= 한 줄씩), 출력은 프린터 한 줄씩 — 컴퓨터의 입출력은 태생부터 **줄 단위의 흐름**이었다.

8.3 프린터 터미널 — 종이 위의 대화, 그리고 tty

시분할의 시대가 오며(여러 사람이 한 컴퓨터를 동시에 쓰는 방식) 사람과 컴퓨터가 **대화**할 장치가 필요해졌다. 그 자리에 들어온 것이 전신 타자기 — **텔레타이프**(Teletype)다. 타자기처럼 생긴 이 기계로 글쇠를 치면 그 글자가 컴퓨터로 가고, 컴퓨터의 응답이 같은 기계의 종이에 찍힌다. 화면은 없다. 대화 전체가 종이 두루마리에 인쇄된다.

Unix가 바로 이 장치들 곁에서 태어났고(2장), 단말 장치를 가리키는 이름을 텔레타이프의 약자에서 따 왔다 — **tty**. 오늘날의 터미널 창 밑바닥에 여전히 살아 있는 이름이다.

타자기의 몸에서 온 유산이 또 있다. 타자기에서 줄을 바꾸는 동작은 사실 두 개다 — 활자 뭉치(캐리지)를 왼쪽 끝으로 **되돌리는** 동작과, 종이를 한 줄 **올리는** 동작. 문자표는 이 두 동작에 각각 제어 문자를

배정했다. 캐리지 리턴(CR, C 표기 `\r`)과 라인 피드(LF, `\n`)다. 줄 하나를 바꾸는 데 지시 두 개가 필요했던 것은 기계 부품이 실제로 둘 움직였기 때문이다.

● CRLF — 타자기가 남긴 백 년짜리 숙제

물리적 이유가 사라진 뒤에도 두 문자는 남았고, 세계는 줄바꿈 표기를 통일하지 못했다. Windows 계열은 두 동작을 다 적는 `\r\n`(CRLF)을, Unix 계열은 `\n` 하나를 줄의 끝으로 삼았다. 그 결과가 오늘날의 일상적 사고들이다 — 한쪽에서 만든 텍스트 파일을 다른 쪽에서 열면 줄 끝에 유령 문자가 붙거나 온 파일이 한 줄로 보이고, 버전 관리 도구(git)는 “CRLF를 LF로 바꿀 것”이라는 경고를 낸다. 타자기 부품의 움직임이, 부품이 사라진 지 반세기 뒤의 협업 현장에서 아직 경고문을 띄우고 있는 셈이다.

8.4 화면 터미널, 그리고 스트림이라는 유산

이윽고 종이가 화면으로 바뀌었다 — 그러나 새 화면 장치들은 프린터 터미널을 **홍내 내는** 방식으로 만들어졌다. 글자가 아래에서 위로 밀려 올라가는 오늘의 터미널 창은, 종이가 위로 밀려 올라가던 텔레타이프의 유리판 버전이다. 겉모습만 바뀌고 대화의 문법은 그대로 남은 것이다.

이 물리적 계보 전체가 하나의 관념으로 응축됐다. 입출력이란 **문자가 한 줄씩, 순서대로 흘러가는** 띠라는 관념 — 이것이 **스트림(stream)**이다. 카드가 한 장씩 빨려 들어가듯 입력이 흘러 들어오고, 프린터가 한 줄씩 찍듯 출력이 흘러 나간다. C는 이 관념을 언어의 입출력 모델로 그대로 받아들였다. 프로그램마다 기본으로 주어지는 **표준 입력**과 **표준 출력**이라는 두 흐름이 그것이고, 이 책의 다음 부에서 만날 `printf`는 바로 표준 출력이라는 띠에 글자를 흘려보내는 함수다.

문 요즘 프로그램은 창과 버튼으로 소통하는데, 스트림은 옛날이야기 아닌가?

답 화면의 앞쪽만 보면 그렇게 보이지만, 뒤쪽에서는 스트림이 오히려 전성기다. 서버 프로그램들의 기록(로그)은 스트림이고, 프로그램과 프로그램을 이어 붙이는 파이프라인도, 인공지능 챗봇이 답을 한 글자씩 “흘려보내는” 것도 스트림이다. 무엇보다 프로그래밍을 배우는 자리에서는 스트림이 가장 단순하고 정직한 소통로다 — 이 책의 예제가 전부 스트림 위에서 도는 이유다.

표현의 사다리가 완성됐다. 사물함에 수를 담는 법(6장), 글자를 담는 법(7장), 그리고 글자를 흘려보내는 법(8장)까지 — 이제 기계에 무엇이든 담고 내보낼 수 있다.

다음 장부터는 이 부의 마지막 등반이다. 지금까지의 단순한 기계 그림이 사실 얼마나 과감한 거짓말이었는지 — 기억이 갈라지고(9장), 실행이 겹쳐지고(10장), 컴파일러가 끼어들면서(11장), C가 왜 “추상적인 언어”일 수밖에 없는지(12장)의 이야기가 남았다.

9 기억의 분화 — 레지스터, 캐시, 다층의 사다리

이 장이 끝나면

고백으로 시작하는 장이다 — 2장의 세 부품 그림이 사실 얼마나 과감한 단순화였는지 밝히고, 그림을 수리한다. CPU 안에 원래부터 있던 레지스터, 바깥의 큰 메모리, 그리고 둘 사이의 속도 격차가 벌어지며 그 틈에 끼어든 다층의 캐시까지 — 기억이 하나가 아니라 사다리라는 것을 알게 된다.

돌아보기 2장에서 “CPU가 메모리에서 다음 할 일을 가져와 그대로 한다”고 했다. 그러면 계산 자체도 메모리 위에서 하는가 — 100번 칸과 104번 칸을 더한 결과가 바로 108번 칸으로 가는가?

답 아니다 — 그리고 이 질문이 이 장의 문을 연다. CPU는 메모리 위에서 직접 계산하지 않는다. 계산은 CPU 안의 별도 공간에서 일어난다. 2장의 그림에는 그 공간이 아예 그려져 있지 않았다. 이제 정직해질 시간이다.

9.1 고백 — 그 그림은 과하게 단순했다

2장의 CPU·메모리·클록 모델은 배움의 첫 계단으로는 훌륭하지만, 오늘의 컴퓨터에 대해서는 **과하게 간소화된** 그림이다. 어느 정도인가 하면 — 그 그림대로라면 오늘의 CPU는 시간 대부분을 **멍하니 기다리며** 보내야 한다. 클록은 반세기 동안 수십만 배 빨라졌는데 메모리는 그만큼 빨라지지 못했기 때문이다. 그런데 실제 컴퓨터는 멍하니 있지 않다. 그 사이에 무슨 장치가 들어섰는지가 이 장과 다음 장의 이야기다. 먼저, 처음부터 그림에서 빠져 있던 것부터 채워 넣는다.

9.2 레지스터 — 원래부터 있던, CPU의 손안

CPU 안에는 **레지스터(register)**라는 작은 기억 장소들이 있다. 처음부터 있었다 — 2장의 그림이 생략했을 뿐이다. 개수는 겨우 수십 개, 크기는 워드 하나씩(3장). 대신 속도는 **즉시**다. 클록 박자 안에서 바로 읽고 쓴다. 손안에 쥔 동전이라 해도 좋다 — 주머니(메모리)보다 가깝고, 꺼내는 데 시간이 걸리지 않는다.

중요한 것은 이것이다. **모든 계산은 레지스터 위에서 일어난다**. 두 수를 더하려면 먼저 메모리에서 레지스터로 가져오고(load), 레지스터끼리 더하고, 결과를 메모리로 되돌린다(store). “100번 칸 + 104번 칸”처럼 보이는 계산의 실제 동선은 언제나 [메모리 → 레지스터 → 계산 → 메모리]다.

문 그렇게 중요한 것이 왜 겨우 수십 개뿐인가? 잔뜩 만들면 안 되나?

답 빠름과 많음은 맞바꿈 관계이기 때문이다. 레지스터가 즉시인 이유는 CPU의 계산 회로 바로 곁에, 가장 비싼 방식으로 만들어져 있어서다. 수를 늘리면 거리가 멀어지고 고르는 시간이 붙어서 “즉시”가 무너진다. 그래서 설계는 “아주 빠른 것을 조금”으로 귀결됐고 — 이 맞바꿈이 이 장 전체를 지배하는 원리다: **빠른 기억일수록 작고, 큰 기억일수록 느리다**.

C의 표면에도 이 지층의 흔적이 있다. C에는 `register`라는 키워드가 있는데 — “이 변수를 되도록 레지스터에 두라”는 옛 시절의 부탁이다. 오늘날에는 컴파일러가 사람보다 배치를 훨씬 잘해서 이 부탁은 장식 이 됐지만, 변수를 누가 레지스터에 올리고 언제 메모리로 내리는가라는 문제 자체는 여전히 살아 있고, 11장(컴파일러 최적화)의 핵심 소재가 된다.

9.3 벌어지는 격차 — 레지스터와 메모리 사이

이제 두 기억을 나란히 세워 보면 그림이 선명해진다.

- **레지스터**: 수십 개 × 워드 크기. 즉시.
- **메모리(DRAM)**: 수백억 칸. 오늘의 CPU 기준으로 수백 클록 박자를 기다려야 답이 온다.

처음부터 이랬던 것은 아니다. C가 태어난 시절(2장)에는 CPU와 메모리의 걸음이 얼추 맞았다. 그런데 이후 수십 년, CPU의 클록은 폭주하듯 빨라졌고 메모리는 커지는 쪽으로 진화하느라 속도가 그만큼 따라 오지 못했다. 격차는 해마다 벌어져 수백 배가 됐다 — 손안의 동전과, 왕복 한 시간짜리 창고의 관계가 된 것이다. 계산은 즉시인데 재료 조달이 수백 박자라면, 기계는 일하는 시간보다 기다리는 시간이 길어진다.

△ “메모리 접근은 어디를 읽든 같은 비용이다”

단순한 그림이 심어 주는 자연스러운 오개념이고, 프로그램의 속도를 이해하려 할 때 가장 먼저 부수어야 할 착각이다. 실제로는 방금 만졌던 근처를 다시 만지는 것과 처음 가 보는 먼 곳을 만지는 것의 비용이 수십 수백 배 다르다 — 왜 그런지가 바로 다음 절의 캐시다. 같은 일을 하는 두 프로그램이 메모리를 어떤 순서로 만지느냐만으로 몇 배씩 차이 나는 이유가 여기에 있다.

9.4 캐시 — 격차의 틈에 끼어든 중간층

수백 배 격차의 해법으로 공학이 내놓은 것이 캐시(cache)다. 착상은 도서관에서 일하는 사람의 요령 그대로다. 서고(메모리)까지 왕복이 느리다면 — **방금 가져온 책을 책상 위에 얼마간 두라**. 다시 찾을 때 책상에 있으면(적중) 즉시고, 없으면(미스) 그때만 서고에 다녀온다.

이것이 통하는 이유는 프로그램의 버릇 때문이다. 프로그램은 메모리를 아무 데나 무작위로 만지지 않는다 — **방금 만진 곳을 곧 다시 만지고** (시간 지역성), **만진 곳의 이웃을 이어서 만진다**(공간 지역성). 루프가 같은 변수를 반복해 만지고, 배열을 앞에서부터 차례로 훑는 것을 떠올리면 된다. 이 버릇 덕에, 작은 책상 하나로도 서고 왕복의 대부분을 없앨 수 있다.

공간 지역성을 활용하는 요령이 하나 더 있다. 캐시는 메모리에서 물건을 **한 칸씩 가져오지 않는다**. 요청한 칸의 이웃까지 묶어 — 오늘날 대개 64바이트, 사물함 예순네 칸을 상자째 — 한 번에 나른다. 이 운반 단위가 **캐시 라인(cache line)**이다. 배열을 차례로 훑을 때 첫 칸에서 한 번만 서고에 다녀오면 다음 예순네 칸이 공짜인 이유이고, 거꾸로 멀리멀리 건너뛰며 만지는 프로그램은 상자를 매번 새로 나르느라 느려지는 이유다. 이 “상자” 개념은 다음 장에서 뜻밖의 사고(false sharing)의 주인공으로 다시 나온다.

9.5 캐시의 분화 — 다층의 사다리

캐시에도 레지스터와 같은 맞바꿈이 적용된다 — 빠르려면 작아야 하고, 크려면 느려진다. 그래서 캐시는 하나가 아니라 **층**으로 분화했다. CPU 바로 곁의 작고 가장 빠른 L1, 그 뒤의 중간 L2, 여러 코어가 함께 쓰는 크고 상대적으로 느린 L3 — 그리고 그 너머의 메모리. 대략의 감각만 숫자로 적으면 이렇다(기계가 다 다르다).

층	크기의 규모	대략의 대기	비유
레지스터	수십 개	0 (즉시)	손안의 동전
L1 캐시	수십 KiB	박자 몇 개	책상 위
L2 캐시	수백 KiB MiB	십수 박자	책장
L3 캐시	수 수십 MiB	수십 박자	같은 층 서가
메모리(DRAM)	수십 GiB	수백 박자	창고

읽는 법은 하나다 — **아래로 한 층 내려갈 때마다 크게 느려진다**. 기억은 “한 덩어리”가 아니라 이 사다리 전체이고, 프로그램의 속도는 상당 부분 “필요한 것이 사다리의 몇 층에서 발견되는가”로 정해진다.

● 같은 일, 다섯 배의 시간 — 2차원 표 훑기

큰 2차원 표(예: 수천×수천의 수)를 전부 더하는 단순한 일을 생각하면 된다. 행을 따라 — 메모리에 놓인 순서 그대로 — 훑으면 캐시 라인 상자가 매번 알뜰하게 쓰인다. 열을 따라 — 매번 멀리 건너뛰며 — 훑으면 상자를 나르고 버리기를 반복한다. 하는 일(덧셈의 횟수)은 완전히 같은데, 실측하면 몇 배에서 수십 배까지 차이가 난다. 코드 두 줄의 순서 차이가 낳는 이 격차는, 기억이 사다리라는 사실을 모르면 설명할 길이 없다 — 그리고 이런 실측을 이 책의 예제로 직접 보게 된다(27장).

문 프로그래머가 캐시를 직접 조종할 수는 없는가 — “이건 L1에 넣어 뒀” 같은 지시는?

답 일반적으로는 없다. 캐시는 하드웨어가 자동으로 운용하고, C 언어에도 캐시를 직접 만지는 표준 문법은 없다. 프로그래머가 하는 일은 조종이 아니라 **협조**다 — 데이터를 이웃끼리 모아 두고, 만질 때 순서대로 만지는 것. 기억이 사다리임을 아는 사람의 코드는 자연스럽게 그렇게 생긴다. 그것으로 충분히, 방금 본 몇 배의 차이가 난다.

그림 수리의 전반부가 끝났다 — 기억은 하나가 아니라 레지스터에서 창고까지의 사다리다. 그러나 아직 절반이다. CPU는 기다림을 줄이는 것으로 만족하지 않고, **실행 자체를 겹치는** 쪽으로도 진화했다 — 조립 라인처럼 명령들을 포개고, 갈림길을 미리 짐작하고, 끝내 일꾼(코어)의 수를 늘렸다. 그 이야기, 그리고 그 요동의 한복판에서 C가 처음으로 계약서(C89)를 쓰게 된 사연이 다음 장이다.

10 속도의 기계장치 ↔ 표준의 탄생

이 장이 끝나면

기다림을 줄인 것이 캐시였다면, 이번에는 실행 자체를 겹치는 장치들이다 — 파이프라인, 분기 예측, 그리고 클럭의 한계가 부른 멀티코어까지. 코어가 여럿이 되며 생기는 기묘한 사고(false sharing)도 구경한다. 나란히, 같은 요동의 시대에 C가 처음으로 계약서(C89)를 쓰게 된 사연을 본다.

돌아보기 2장의 문답에서 “걸음 수(클럭)가 전부는 아니다 — 한 걸음에 여러 일을 하는 요령이 현대 CPU의 진짜 무기”라고 예고했다. 한 걸음에 여러 일이란 어떻게 가능한가? 한 명령이 끝나야 다음 명령을 하는 것 아닌가?

답 “끝나야 시작한다”를 버리면 된다. 명령 하나의 처리를 여러 단계로 쪼개고, 단계가 다른 여러 명령을 **동시에** 진행하는 것이다 — 한 명령이 계산되는 동안 다음 명령을 가져오고, 그다음 명령을 해독한다. 이 장의 첫 장치, 파이프라인이다.

10.1 파이프라인 — 조립 라인 위의 명령들

세탁소를 떠올리면 된다. 세탁 30분, 건조 30분, 다림질 30분이라 할 때, 한 벌을 끝내고 다음 벌을 시작하면 세 벌에 270분이 걸린다. 그러나 첫 벌이 건조기로 넘어가는 순간 둘째 벌을 세탁기에 넣으면 — 장비 세 대가 **서로 다른 벌을 동시에** 처리하며, 흐름이 차오른 뒤에는 30분마다 한 벌씩 완성되어 나온다.

CPU가 명령을 처리하는 것도 똑같이 단계로 나뉜다 — 가져오고(fetch), 해독하고(decode), 계산하고(execute), 결과를 쓴다(write). 이 단계들을 조립 라인으로 만든 것이 **파이프라인(pipeline)**이다. 명령 하나하나가 빨라지는 것이 아니다 — **단위 시간에 완성되는 명령의 수가** 늘어난다. 현대 CPU의 파이프라인은 열 단계가 넘고, 여기에 라인을 여러 개 두어 박자당 여러 명령을 완성하는 데까지 나아갔다. 2장의 “한 박자에 한 걸음”은 이렇게 무너진다 — 걸보기 순서는 지키되, 속은 겹쳐 돌아간다.

같은 시대의 요령 하나를 여기 적어 둔다. 루프 한 바퀴마다 드는 관리 비용(횃수 세기, 갈림길 판단)을 아끼려고 **한 바퀴에 여러 개씩** 일을 처리하는 기법이 있다 — 루프 언롤링(loop unrolling)이라 한다. 1983년, Tom Duff라는 프로그래머는 이 기법을 C의 두 문법을 기괴하게 겹쳐 쓰는 방법으로 극한까지 밀어붙였고, 그 코드는 **Duff's device**라는 이름의 전설이 됐다. 코드 자체는 아직 보여 줄 수 없다 — 이 책이 그 문법들을 아직 소개하지 않았기 때문이다. 재회를 예약해 둔다(27장). 지금은 한 문장이면 된다: 그 시절에는 사람이 이런 곡예로 기계의 박자를 짜냈다는 것, 그리고 오늘날 그 곡예는 컴파일러의 일이 됐다는 것(11장).

10.2 분기 예측 — 갈림길을 미리 짐작하다

파이프라인에는 아픈 곳이 있다. **갈림길**이다. “결과가 0이면 저쪽으로 건너뛰어라” 같은 명령(분기)을 만나면, 어느 쪽 길의 명령을 라인에 실어야 할지 결과가 나오기 전에는 모른다. 라인을 세우고 기다리면 겹침의 이득이 사라진다.

그래서 CPU는 **짐작**한다. 단골 손님's 주문을 미리 만들어 두는 카페처럼, 지금까지의 이력으로 “이 갈림길은 대개 이쪽”을 학습해 그쪽 명령을 미리 라인에 싣는다 — **분기 예측(branch prediction)**이다. 맞으면 공짜고, 틀리면 미리 만든 주문을 전부 버리고 라인을 비운 뒤 다시 시작한다. 이 벌금이 파이프라인 깊이만큼 크기 때문에, 현대 CPU는 예측기에 진지한 회로를 투자하고 적중률은 일상 코드에서 90%를 훌쩍 넘는다.

● 정렬된 배열이 더 빨랐던 사건

프로그래머들 사이에 널리 회자된 실측이 있다 — 큰 배열에서 “값이 일정 기준보다 큰 것만 더하라”는 같은 코드가, 배열을 **미리 정렬해 두면** 몇 배 빨라진다는 관찰이다. 덧셈의 횟수는 같은데 왜인가. 답은 분기 예측이다. 뒤섞인 배열에서는 “크다/작다” 갈림길이 매번 무작위라 예측이 반타작으로 틀리고, 그때마다 라인을 비우는 벌금을 문다. 정렬된 배열에서는 갈림길의 답이 한동안 같은 쪽으로 이어져 예측이 거의 다 맞는다. 눈에 보이지 않는 짐작 회로 하나가, 같은 코드의 속도를 몇 배 가른 것이다 (26장에서 이 감각을 다시 쓴다 — “if 는 공짜가 아니다”).

10.3 클록의 한계, 그리고 멀티코어

이 모든 겹침에도 불구하고, 2000년대 중반에 벽이 왔다. 클록을 더 올리면 전력과 발열이 감당할 수 없이 치솟는 물리적 한계였다. 수십 년 이어지던 “내년에는 더 빠른 클록”이 멈춘 것이다.

산업의 응답은 방향 전환이었다 — 한 일꾼을 더 빠르게 만드는 대신, **일꾼의 수를 늘린다**. CPU 하나에 독립적으로 명령을 실행하는 엔진 — **코어(core)** — 를 두 개, 네 개, 수십 개씩 넣기 시작했다. 오늘날의 스마트폰조차 코어 여덟 개쯤은 예사다. 공짜 점심은 끝났다 — 프로그램이 저절로 빨라지던 시대가 끝나고, 여러 일꾼에게 일을 **나누어 주는** 문제가 프로그래머의 몫이 된 것이다(이 책의 범위 밖이지만, 지형은 알아 둔다).

그리고 일꾼이 여럿이 되자, 9장의 캐시가 뜻밖의 사고 현장이 된다.

문 코어마다 자기 캐시(책상)가 있다면, 두 코어가 같은 데이터를 만질 때는 무슨 일이 일어나는가?

답 하드웨어가 뒤에서 책상들 사이의 일관성을 맞춰 준다 — 한 코어가 값을 고치면 다른 코어의 책상에 놓인 사본을 무효로 만드는 식이다. 정확성은 이렇게 지켜지는데, 문제는 이 정리 정돈이 **공짜가 아니라는 것**, 그리고 그 단위가 변수 하나가 아니라 9장의 **캐시 라인 상자**라는 것이다. 여기서 기묘한 사고가 태어난다.

False sharing(거짓 공유)이라 불리는 사고다. 두 코어가 **서로 다른** 변수를 만지는데 — 공유한 것이 하나도 없는데 — 하필 두 변수가 **같은 캐시 라인 상자**에 나란히 놓여 있으면, 하드웨어의 눈에는 같은 상자를 두 코어가 다투는 것으로 보인다. 한쪽이 쓸 때마다 다른 쪽 책상의 상자가 무효가 되고, 상자가 두 책상 사이를 핑퐁처럼 오가며, 두 코어 모두 수십 배 느려진다. 코드만 보면 아무것도 공유하지 않으므로 원인이 보이지 않는다 — 기억이 상자 단위로 움직인다는 9장의 사실을 알아야만 진단이 되는, 현대적인 사고의 대표 사례다.

10.4 같은 시대, C는 계약서를 썼다 — 표준 이전의 C와 C89

기계가 이렇게 요동치던 시기, C의 세계에도 다른 종류의 요동이 있었다.

C에는 오랫동안 **공식 표준이 없었다**. 1978년에 나온 K&R의 책 “The C Programming Language”가 **사실상의 표준** 노릇을 했다 — 언어의 규격서가 아니라, 잘 쓰인 한 권의 책이 법전을 대신한 것이다. Unix와 함께 C가 대학과 업계로 번지자 회사마다 자기 컴파일러를 만들었고, 책이 못박지 않은 구석마다 방언이 자랐다. 그 시절의 C는 지금 눈에는 험잡다 — 함수를 선언할 때 인자의 타입을 적지 않았고(원형이 없었다), 타입을 안 적은 이름은 슬그머니 int로 통했다. 어느 컴파일러에서 되던 코드가 다른 컴파일러에서 안 되는 일이 예사였고, 이식은 번번이 모험이었다.

건디다 못한 산업이 1983년 표준화 위원회(ANSI X3J11)를 꾸렸고, 6년의 격론 끝에 1989년 **C89** — 최초의 공식 C 표준 — 가 나왔다. 함수 **원형**(인자 타입까지 적는 선언, 21장)을 들여 컴파일러가 호출의 실수를 잡을 수 있게 했고, 험거운 구석들을 조였으며, 무엇보다 “프로그래머는 무엇을 약속받고, 구현은 어

디까지 자유로운가”를 처음으로 **문서**로 못박았다. 6장의 IEEE 754(1985)와 같은 몇 해의 일이다 — 하드웨어의 수도, 언어의 문법도, 벤더마다 제멋대로이던 것을 계약서로 다스리기 시작한 “계약의 시대”였다.

이 계약이라는 관념이 왜 갈수록 중요해지는가 — 기계는 이 장에서 본 것처럼 갈수록 겹치고 짐작하고 나뉘는데, 프로그래머가 그 소용돌이를 일일이 알 수는 없다. 소용돌이와 프로그래머 사이에 **또 하나의 층**이 서 있기 때문이다. 소스 코드를 받아 기계의 말로 옮기면서, 뜻만 지킨다면 무엇이든 뜯어고칠 권리를 가진 층 — 컴파일러다. 다음 장의 주인공이다.

11 컴파일러 최적화 ↔ 추상 기계

이 장이 끝나면

컴파일러가 소스 코드를 “그대로” 번역하지 않는다는 것 — 뜻만 지키면 마음껏 재배열하고 지우고 바꿔 쓴다는 것 — 을 알게 된다. 그 자유의 근거인 “관찰 가능한 결과”라는 개념과, 표준 문서 속의 가상 기계인 추상 기계를 만난다. C99·C11이 이 계약을 어떻게 정밀하게 다듬었는지도 본다.

돌아보기 10장 끝에서 컴파일러를 “뜻만 지킨다면 무엇이든 뜯어고칠 권리를 가진 층”이라 불렀다. 그런데 3장에서 C는 “기계의 정직한 별명”이라 하지 않았나 — 내가 쓴 C 코드는 기계 명령에 그대로 대응하는 것 아니었나?

답 그 시절에는 대체로 그랬고, 지금은 아니다. 오늘의 컴파일러는 직역가가 아니라 **편집자**다 — 뜻을 지키는 한에서 문장을 통째로 다시 쓴다. “그대로 대응”이라는 초기 C의 감각이 어떻게, 왜 무너졌는가가 이 장이고, 그 무너짐이야말로 C를 추상적인 언어로 만든 마지막 조각이다.

11.1 편집자의 작업실 — 컴파일러가 하는 일

몇 가지 실제 편집 사례를 보면 감각이 잡힌다. 컴파일러는 일상적으로 이런 일을 한다.

미리 계산한다. 소스에 $\text{초} = 24 * 60 * 60$ 이라 적혀 있으면, 곱셈 명령을 만들지 않고 그냥 86400을 적어 넣는다. 실행 시점에 곱할 이유가 없기 때문이다.

죽은 일을 지운다. 계산해 놓고 아무도 안 쓰는 값, 도달할 수 없는 갈림길 — 통째로 사라진다. 소스에 있는 코드가 실행 파일에는 존재하지 않을 수 있다.

순서를 바꾼다. 서로 상관없는 계산이라면, 파이프라인(10장)이 잘 차도록 앞뒤를 재배열한다. 소스의 줄 순서는 실행 순서의 약속이 아니게 된다.

메모리 왕복을 없앤다. 루프가 도는 내내 변수 하나를 매번 메모리에서 읽고 쓰는 대신, 레지스터(9장)에 올려 두고 끝날 때 한 번만 내려놓는다. 소스에는 천 번의 메모리 접근이 적혀 있어도 실제로는 두 번 일 수 있다.

이 편집들이 겹치고 쌓이면, 실행 파일 속 기계 명령과 소스 코드는 문장 대 문장으로 대응하지 않는 별개의 문서가 된다. 디버거로 최적화된 프로그램을 들여다보면 “이 줄은 최적화로 사라졌음”이라는 안내를 만나는 것이 그래서다.

문 몇대로 고쳐도 되는 근거가 무엇인가? 어디까지 고쳐도 “뜻을 지켰다”고 치는가?

답 바로 그 경계선을 긋는 것이 표준의 일이고, 경계선의 이름이 **관찰 가능한 결과**다. 대략 이렇다 — 프로그램이 바깥세상과 주고받는 것(출력한 내용, 읽은 입력, 특별히 보호된 접근)만 약속대로면, 그 결과를 만들어 내는 속사정은 전부 컴파일러의 재량이다. 중간 계산을 몇 번 하는지, 변수가 메모리에 실제로 있는지, 어떤 순서로 일하는지 — 바깥에서 관찰되지 않는 한 아무래도 좋다. 겉보기만 같으면 속은 자유 — 이것이 계약의 문장이다.

11.2 추상 기계 — 계약서 속의 가상 기계

그러면 “겉보기”의 기준은 누가 정하는가. 실제 기계일 수는 없다 — 실제 기계는 수천 종이고 저마다 다르다. 그래서 표준은 문서 안에 **가상의 기계**를 하나 정의해 두었다. C 프로그램의 의미란 “이 가상 기계 위에서 이렇게 실행되는 것”이라고 못박는, 종이 위에만 존재하는 기계 — **추상 기계**(abstract machine)다.

이제 세 층의 관계가 선명해진다.

- **프로그래머**는 추상 기계를 상대로 코드를 쓴다. 코드의 의미는 추상 기계 위에서의 실행이 정한다.
- **컴파일러**는 그 의미의 관찰 가능한 결과만 지키면서, 실제 기계에 맞는 가장 빠른 코드를 자유롭게 지어 낸다.
- **실제 기계**는 캐시가 몇 층이든 파이프라인이 몇 단이든(8·10장), 그 소용돌이를 겉으로 드러내지 않는다.

3장에서 “C는 기계의 정직한 별명”이라는 평판이 오늘날 절반만 맞다고 했던 것의 정답이 이것이다. C가 별명 노릇을 해 주는 대상은 이제 실제 기계가 아니라 **추상 기계**다. 실제 기계와의 대응은 컴파일러가 그 때그때 지어내는, 겉보기만 보장된 번역이다.

11.3 계약의 정밀화 — C99, C11

C89가 계약서의 초판이었다면(10장), 이후의 표준들은 조항을 정밀하게 다듬어 온 개정판들이다. 기계와 컴파일러가 공격적으로 진화할수록 “어디 까지 고쳐도 되는가”를 더 날카롭게 그어야 했기 때문이다. 두 번의 큰 개정만 짚는다.

C99(1999)는 편집자에게 더 많은 재량 근거를 주는 쪽으로 언어를 다듬었다 — 대표적으로, 타입이 다른 이름끼리는 같은 기억을 가리지 않는다고 **전제해도 좋다**는 규칙(엄격한 앨리어싱)이 명문화됐다. 이 전제가 있어야 편집자가 “이 둘은 서로 무관하다”고 보고 재배열·캐싱을 할 수 있다. 대가는 프로그래머 쪽 의무다 — 그 전제를 깨는 코드(같은 기억을 다른 타입의 눈으로 읽는 일)는 계약 위반이 된다(38장에서 안전한 방법과 함께 재론한다).

C11(2011)은 10장의 멀티코어 시대에 대한 응답이다 — 일꾼이 여럿일 때 “한 코어의 쓰기가 다른 코어에 언제 보이는가”를 계약 조항으로 못박은 **메모리 모델**이 표준에 들어왔다. 그 전까지 다중 코어 위의 C는 표준 바깥의 관행에 기대던, 말 그대로 계약서 없는 동업이었다.

● 사라진 지우개 — 최적화가 지운 보안 코드

“겉보기만 같으면 속은 자유”가 실제로 문 사고가 있다. 보안 프로그램들은 비밀번호를 다 쓴 뒤 그 메모리를 0으로 덮어 지우는 관행이 있다 — 메모리 어딘가에 비밀이 남아 있지 않도록. 그런데 편집자의 눈에 이 지우기는 **아무도 다시 읽지 않는 쓰기**, 즉 죽은 일이다. 관찰 가능한 결과에 아무 기여가 없으므로 — 지워도 된다. 실제로 여러 컴파일러가 이 지우기를 조용히 삭제했고, 비밀번호가 메모리에 남은 채 배포된 소프트웨어들이 보안 권고의 대상이 됐다. 계약을 무시한 쪽은 컴파일러가 아니라, 계약서를 안 읽은 사람이었다 — 이후 표준과 플랫폼들은 “이 쓰기는 지우지 말라”고 명시하는 장치(memset_s 등)를 마련했다. 관찰 가능한 결과라는 조항 하나가 실무에서 얼마나 무거운지 보여주는 사건이다.

문 최적화가 이렇게 위험하면, 그냥 끄고 살면 안 되는가?

답 개발 중의 디버깅 빌드는 실제로 최적화를 낮춰서 소스와 실행의 대응을 살려 둔다. 그러나 배포에서 끄는 것은 답이 아니다 — 현대 소프트웨어의 속도는 상당 부분 편집자의 숨씨에서 나오고, 몇 배의 성능을 포기할 이유는 없다. 바른길은 하나다: 계약서를 알고, 계약 안에서 쓰는 것. “내 컴퓨터에서 돌아다”가 아니라 “추상 기계 위에서 옳다”가 기준이 되어야 한다 — 계약 밖의 코드에 무슨 일이 벌어지는지는 41장(정의되지 않은 동작)에서 정면으로 다룬다.

이제 재료가 다 모였다. 기억은 사다리이고(9장), 실행은 겹침과 짐작이며(10장), 그 소용돌이 위에 편집자가 서서 겉보기만 보장한다(11장). 이 셋을 합치면 하나의 결론이 나온다 — C 프로그래머가 상대하는

것은 실제 기계가 아니라는 것. 다음 장에서 이 부의 논제를 완성하고, C가 결국 **어느** 기계의 언어인지 — CPU와 GPU의 대조까지 — 파노라마로 마무리한다.

12 그래서 C는 추상적인 언어다

이 장이 끝나면

제2부의 결론이다. 흩어져 있던 조각 — 사다리가 된 기억, 겹쳐 도는 실행, 편집자 노트를 하는 컴파일러 — 을 모아 하나의 명제로 완성한다: C는 기계어의 별명이 아니라 추상 기계의 언어이고, 그래서 추상적으로 접근해야 한다. 요즘 더 엄격해지는 포인터 규칙(프로버넌스)이 왜 그 귀결인지 보고, 마지막으로 CPU와 GPU를 대조하며 C가 “어느 기계의” 언어인지로 부를 닫는다.

돌아보기 11장에서 “C 프로그래머가 상대하는 것은 실제 기계가 아니다”라고 했다. 그러면 2장에서 3장 내내 실제 기계 — 클록, 사물함, 워드 — 를 그렇게 들여다본 것은 헛수고였는가?

답 반대다. 그 그림들이 바로 추상 기계의 **뼈대**다. 표준의 추상 기계는 허공에서 지어낸 것이 아니라 실제 기계들의 공통 뼈대 — 바이트 열의 메모리, 주소, 순차 실행 — 를 추려 계약서에 옮겨 적은 것이다. 실제 기계를 알아야 추상 기계가 왜 그 모양인지 이해되고, 추상 기계를 알아야 실제 기계의 소용돌이(8 11장)에 흔들리지 않는다. 두 그림은 경쟁자가 아니라 한 벌이다.

12.1 논제의 완성 — 세 조각을 합치다

이 부가 쌓아 온 조각을 한 줄씩으로 줄이면 이렇다.

- 9장 — 기억은 한 덩어리가 아니라 레지스터에서 창고까지의 **사다리**이고, 내 변수가 지금 어느 층에 있는지 소스 코드에는 적혀 있지 않다.
- 10장 — 실행은 한 걸음씩이 아니라 **겹침과 짐작**이고, 명령들의 실제 진행 순서는 소스의 줄 순서와 다르다.
- 11장 — 그 위에 **편집자**가 서서, 겉보기 결과만 지키며 코드를 통째로 다시 쓴다.

그러면 소스 코드의 한 줄 한 줄이 “기계가 그대로 할 일”이라는 생각은 세 겹으로 무너진다. 남은 단단한 것은 하나 — **계약**이다. 표준이라는 계약서가 정의한 가상의 기계(추상 기계) 위에서 내 코드가 무엇을 뜻하는가. 그것만이 C 프로그램의 진짜 의미이고, 나머지 전부 — 캐시 적중, 파이프라인, 재배열, 레지스터 배치 — 는 그 의미를 빠르게 실현하는 속사정이다.

그래서 C는 추상적인 언어이고, 추상적으로 접근해야 한다. “이 코드를 기계가 어떻게 돌릴까”를 짐작하는 습관이 아니라, “이 코드가 계약 위에서 무엇을 뜻하는가”를 묻는 습관 — 이 책이 이후 모든 장에서 기르려는 것이 바로 그 습관이다.

△ “C는 하드웨어를 직접 조작하는 언어다”

절반의 역사와 절반의 오해가 섞인, C에 관한 가장 유명한 문장이다. 역사적으로는 사실에 가까웠고(2·3장), 지금도 C는 하드웨어에 **가장 가까운 곳까지 데려다주는** 언어다. 그러나 오늘의 C 코드와 하드웨어 사이에는 편집자(컴파일러)와 계약서(표준)가 서 있고, 순진한 “직접 조작” 감각으로 쓴 코드 — 이를테면 계약 밖의 지름길 — 는 편집자의 재량 앞에서 부서진다(11장의 사라진 지우개, 그리고 41장). 정확한 문장은 이렇다: C는 하드웨어를 직접 조작하는 언어가 아니라, **하드웨어를 추상 기계라는 계약으로 만나게 해 주는 언어**다.

12.2 요즘의 흐름 — 포인터에 붙는 출처

이 관점이 요즘 어디로 가고 있는지 보여 주는 대표적인 흐름이 포인터 규칙의 정밀화다.

3장에서 “주소도 수”라고 했다. 순진한 그림에서 포인터는 그냥 번호다 — 같은 번호면 같은 곳이고, 번호만 맞으면 어떻게 얻었든 그곳을 만질 수 있어야 한다. 그러나 편집자의 세계에서 이 그림은 유지될 수

없다. 컴파일러의 최적화는 “이 포인터는 어디에서 왔는가”라는 계보 추적 위에 서 있기 때문이다 — 이 변수에서 나온 포인터는 저 변수를 건드릴 수 없다는 전제(11장의 엄격한 앨리어싱도 그 한 갈래)가 있어야 재배열과 캐싱이 가능하다.

그래서 현대의 표준 논의는 포인터를 “번호”가 아니라 “번호 + 출처 딱지”로 다듬어 가고 있다. 같은 비트 패턴이라도 어디에서 유래했는가에 따라 쓸 수 있는 곳이 다르다는 것 — 이 출처 개념을 **프로버넌스**(provenance)라 부르고, C 표준 위원회는 이를 명문화하는 작업을 진행해 왔다. 방향은 일관된다: 포인터에 대한 규칙이 점점 더 명시적이고 엄격해지는 쪽, 즉 계약서의 조항이 더 정밀해지는 쪽이다. 입문 단계에서 조항의 세부까지 알 필요는 없다 — “포인터는 그냥 수가 아니다, 계보가 있다”는 감각만 여기서 심고, 실무 규칙은 31장에서, 계약 위반의 결과는 41장에서 만난다.

문 4장의 태그 포인터 — 주소의 빈 비트에 딱지를 끼우는 재주 — 도 이 규칙과 부딪히는가?

답 부딪힐 수 있는 지점이 정확히 그곳이다. 포인터를 수처럼 주물러 태그를 끼우고 떼는 일은 “번호 = 포인터” 그림에서는 자명했지만, 출처 딱지의 세계에서는 **계보를 잃지 않는 절차**를 지켜야 하는 일이 된다(정수로 변환해 다루는 합법적 경로가 마련되어 있다). 저수준 기법일수록 계약서를 정독해야 하는 시대 — 그것이 모던 C다.

12.3 마무리 파노라마 — C는 어느 기계의 언어인가

부를 닫기 전에, 시야를 한 번 넓힌다. “기계”라고 불러 온 것이 사실 한 종류가 아니기 때문이다.

같은 트랜지스터로 정반대의 기계를 지을 수 있다. 하나는 **한 가지 일을 최대한 빨리 끝내는** 기계 — 깊은 파이프라인, 큰 캐시, 정교한 분기 예측(8·10장)이 전부 이것, 즉 **지연시간**(latency)을 줄이는 장치다. 우리가 여태 그려 온 CPU다. 다른 하나는 **같은 일을 산더미에 한꺼번에 하는** 기계 — 예측도 깊은 파이프라인도 덜어 내고, 그 자리에 단순한 계산기를 수천 개 심는다. **처리량**(throughput)의 기계, GPU다. 화면의 수백만 픽셀에 같은 계산을 뿌리려고 태어났고, 그 체질이 행렬 계산 — 인공지능의 심장 — 과 맞아 떨어지며 시대의 주인공이 됐다.

식당으로 비유하면, CPU는 어떤 주문이든 즉시 쳐내는 달인 셰프 한두 명이고, GPU는 같은 메뉴 만 그릇을 동시에 만드는 급식 라인이다. 어느 쪽이 낫냐는 물음은 성립하지 않는다 — 주문이 다르다.

C와의 관계가 이 장의 마지막 문장이 된다. GPU의 세계조차 그 프로그래밍 언어(CUDA, OpenCL)는 C의 방언으로 만들어졌다 — 시스템 세계의 공용어가 C라는 1장의 이야기가 여기서도 반복된다. 그러나 정확히 말하면, **C의 추상 기계는 CPU의 상(像)이다**. 순차 실행, 하나의 메모리, 주소 — 계약서의 가상 기계는 CPU의 뼈대를 본떴고, GPU라는 다른 체질은 그 계약서의 방언과 확장으로 다룬다. C를 배운다는 것은 컴퓨팅의 여러 기계 가운데 가장 근본이 되는 한 상 — 폰 노이만 이래의 순차 기계 — 을 그 언어와 함께 배우는 일이다.

12.4 제2부를 닫으며

바닥 공사가 끝났다. 비트에서 시작해(2장) 사물함 복도를 걷고(3·4장), 정수와 소수와 글자와 흐름을 담는 법을 배웠고(5·8장), 기계가 복잡해진 사연과 그 위에 선 계약을 보았다(9·12장). 이 모든 것이 심화 문답으로 계속 소환될 밑천이다.

이제 정말로 시작할 수 있다. 다음 부에서 드디어 첫 프로그램을 쓴다 — 지면 위에 헬로 월드가 찍히고, 그 한 편을 읽어 내는 여정이 책의 나머지 전부다.

제3부 — 첫 프로그램

13 헬로 월드

이 장이 끝나면

드디어 첫 프로그램이다. 여섯 줄짜리 C 프로그램 한 편을 지면에서 실행하고, 한 줄 한 줄 소리 내어 읽는다. 아직 다 이해되지 않는 것이 정상이다 — 어디까지가 “주문”이고 언제 풀리는지를 여기서 정해 둔다.

돌아보기 7장에서 컴퓨터의 입출력은 “문자가 한 줄씩 흐르는 띠”(스트림)라 했고, 프로그램마다 표준 출력이라는 흐름이 기본으로 주어진다고 했다. 그러면 프로그램이 화면에 글자를 내보낸다는 것은 정확히 무엇을 하는 일인가?

답 표준 출력이라는 띠에 글자들을 흘려보내는 일이다. 화면은 그 띠의 끝에 붙어 있는 오늘날의 “종이”일 뿐이다(유리 텔레타이프의 후예라는 것을 기억하면 된다). 지금 만날 첫 프로그램이 하는 일이 정확히 이것 — 표준 출력에 글자 일곱 개와 줄바꿈 하나를 흘려보내는 것 — 이다.

13.1 첫 프로그램

전통에 따라, 첫 프로그램은 세상에 인사를 건넨다.

```
examples/ch13/hello.c
#include <stdio.h>

int main(void)
{
    printf("안녕, 세상!\n");
    return 0;
}
```

실행 결과

안녕, 세상!

위쪽 상자가 사람이 적는 소스 코드이고, 아래 검은 상자가 이 프로그램을 실제로 컴파일해 실행한 출력이다 — 이 책의 모든 실행 결과가 그렇듯, 장식이 아니라 기계가 실제로 낸 결과다.

13.2 한 줄씩 읽기

`#include <stdio.h>` — 준비물 선언이다. “표준 입출력(`st*an*d*ard i*input`)

14 일반적인 컴파일 과정

이 장이 끝나면

hello.c라는 텍스트가 실행 파일이 되기까지의 네 단계 릴레이 — 전처리, 컴파일, 어셈블, 링크 — 를 구경한다. 어느 플랫폼, 어느 컴파일러(gcc·clang)에서든 통하는 일반적인 그림이다. 13장의 “주문” 두 개(#include의 정체, printf의 출처)가 여기서 풀린다.

돌아보기 2장에서 추상화의 층계 — 트랜지스터에서 언어까지 겹겹이 쌓기 — 를 이야기했다. 그러면 컴파일러는 그 층계에서 몇 층부터 몇 층까지를 내려가는 번역인가?

답 “C 언어” 층에서 “기계 명령” 층까지다. 그런데 이 번역은 한 번에 건너뛰는 도약이 아니라, 중간 층계참을 밟아 내려가는 릴레이다 — 텍스트를 다듬는 층, C를 어셈블리라는 사람이 읽을 수 있는 기계어 표기로 바꾸는 층, 그것을 진짜 기계 명령으로 찍는 층, 그리고 조각들을 한 덩어리로 잇는 층. 이 장이 그 네 층계참의 견학이다.

14.1 겉보기 — 명령 한 줄

먼저 겉모습부터. gcc나 clang 같은 컴파일러에게 소스 파일을 주면 실행 파일이 나온다.

```
$ cc hello.c -o hello      (cc 자리는 gcc 또는 clang)
$ ./hello
안녕, 세상!
```

-o hello는 “결과물의 이름을 hello로 하라”는 뜻이다. 명령 한 줄로 끝나 보이지만, 이 한 줄 뒤에서 서로 다른 일꾼 네 명이 릴레이를 뛴다. 컴파일러에게 “중간 결과를 보여 달라”고 부탁하는 선택지들(-E, -S, -c)로 각 주자를 세워서 구경할 수 있다.

14.2 1주자 — 전처리: 텍스트를 짜깁기한다

전처리기(preprocessor)는 아직 C를 모른다. 그저 텍스트를 다루는 가위와 풀이다. #으로 시작하는 줄들 — 13장의 주문 — 이 전처리기에게 내리는 지시다.

#include <stdio.h>의 정체가 여기서 풀린다. 지시의 뜻은 놀랄 만큼 우직하다 — “stdio.h라는 파일을 찾아서, 그 내용물을 이 자리에 통째로 붙여 넣어라.” 그게 전부다. 실제로 전처리만 시켜 보면(cc -E hello.c) 여섯 줄짜리 hello.c가 수만 줄로 불어난 결과가 나온다 — printf를 비롯한 표준 도구들의 선언(이런 이름의 함수가 존재한다는 예고 — 정식으로서는 21장)이 줄줄이 붙어 들어온 것이다.

전처리기는 이 밖에도 글자 바꿔치기(매크로), 조건에 따라 코드 일부를 넣고 빼기 같은 텍스트 작업을 한다. 6장에서 만난 삼중자를 풀어 주던 것도(그 시절에는) 이 단계였다 — 전처리기는 C 컴파일의 첫 주자이자, 역사의 흥터가 모이는 곳이기도 하다.

14.3 2주자 — 컴파일: C를 어셈블리로

이제부터가 본편이다. 컴파일러 본체가 전처리된 C 코드를 읽고 — 문법을 검사하고, 뜻을 해석하고, 11장의 편집(최적화)을 가한 뒤 — 어셈블리라는 표기로 옮긴다. 어셈블리는 2장에서 스친 그 낮은 층의 언어다: 기계 명령과 거의 일대일이되, 사람이 읽을 수 있게 이름이 붙어 있다. cc -S hello.c로 이 중간 결과(hello.s)를 구경할 수 있는데, 대략 이런 모습의 조각이 들어 있다(기계와 컴파일러마다 다르다):

```
lea    rdi, [rip + .L.str]    ; 인사말의 주소를 준비하고
call   printf                 ; printf를 부른다
xor     eax, eax               ; 0을 만들어
ret                                ; 돌려주며 퇴장
```

우리가 쓴 네 문장의 골격이 그대로 비친다 — 그러나 11장에서 배웠듯, 이 대응이 언제나 이렇게 얹전히 남는 것은 아니다(최적화를 켜면 문장 대 문장 대응은 무너진다).

14.4 3주자 — 어셈블: 기계 명령으로 찍어 내다

어셈블러가 어셈블리 텍스트를 진짜 기계 명령 — 비트들 — 로 찍어 낸다. 결과물이 **목적 파일**(object file, cc -c로 만들면 hello.o)이다. 이제 내용물은 사람이 읽는 텍스트가 아니라 2장의 세계, 기계 명령의 비트 열이다.

그런데 목적 파일은 아직 실행할 수 없다. **구멍이 뚫려 있기 때문이다** — call printf라고 찍기는 했는데, 정작 printf라는 것이 어디에 있는지 이 파일은 모른다. 우리 소스에는 printf의 선언(존재 예고)만 들어왔지, 그 몸통(실제 코드)은 들어온 적이 없다.

14.5 4주자 — 링크: 조각을 잇고 구멍을 메운다

마지막 주자 **링커**(linker)가 그 구멍을 메운다. printf의 몸통은 **표준 라이브러리**라는, 시스템에 미리 컴파일되어 놓여 있는 목적 파일들의 꾸러미 안에 있다. 링커는 우리의 목적 파일과 그 꾸러미를 이어 붙이고, “printf는 여기”라고 주소를 채워 넣어, 비로소 **실행 파일** 하나를 완성한다. 13장의 두 번째 주문 — printf는 어디서 오는가 — 의 답이 이것이다: 내가 쓴 적 없는 코드가, 링크 단계에서 내 프로그램에 이어져 들어온다.

● 가장 낮은 오류 — undefined reference

초보 시절 가장 어리둥절한 오류가 링크 단계에서 나온다. 함수 이름의 철자를 틀리거나 라이브러리를 빠뜨리면, 컴파일은 멀쩡히 통과하고 마지막에 undefined reference to ...(정의를 찾을 수 없음) 같은 메시지가 나온다. “컴파일러는 통과시켰는데 왜?”가 어리둥절함의 정체인데 — 이제 답할 수 있다. 컴파일 단계는 **선언**(예고)만 있으면 만족하고, 몸통을 실제로 찾아 잇는 것은 링커의 일이라서다. 오류를 낸 일꾼이 다르면 오류의 종류도 다르다 — 메시지를 읽을 때 “네 주자 중 누가 넘어졌는가”부터 가리는 습관이 문제 해결의 절반이다(정식은 42장).

△ “컴파일러가 소스를 실행 파일로 한 번에 번역한다”

겉보기(명령 한 줄)가 심어 주는 자연스러운 그림이지만, 실제로는 성격이 전혀 다른 네 도구의 릴레이다 — 텍스트 짜깁기(전처리), 진짜 번역과 편집(컴파일), 비트로 찍기(어셈블), 조각 잇기(링크). 이 구별이 실무 감각의 뿌리가 된다: 오류가 어느 단계의 것인지 가려 읽게 되고, 여러 소스 파일을 각각 목적 파일로 만들어 두었다가 링크만 다시 하는 큰 프로젝트의 빌드 방식(42장)이 자연스러워지고, “선언과 정의”(21장)라는 언어 규칙이 왜 존재하는지 — 컴파일러는 예고만 보고 일하고 링커가 실물을 잇기 때문 — 가 이해된다.

문 네 단계를 매번 직접 시켜야 하는가?

답 아니다 — 보통은 cc hello.c -o hello 한 줄이면 컴파일러가 네 주자를 알아서 차례로 뛰게 한다. 단계 선택지(-E, -S, -c)는 학습과 진단용 도구다. 다만 큰 프로젝트에서는 3주자까지만 파일별로 뛰게 해 두고(각각 .o로) 마지막 링크만 다시 하는 방식이 표준 관행이 된다 — 파일 하나 고쳤다고 전부를 다시 번역할 이유가 없기 때문이다. 그 세계는 42장에서 열린다.

이제 소스에서 실행까지의 지도가 생겼다. 남은 것은 이 릴레이를 **내 컴퓨터에서** 돌릴 수 있게 도구를 갖추는 일이다 — 다음 장에서 컴파일러를 설치하고, 버그를 그물처럼 걸러 주는 현대적 보조 도구(새니타이저)까지 장만한다.

15 개발환경 구축

이 장이 끝나면

14장의 릴레이를 내 컴퓨터에서 돌릴 도구를 갖춘다. 어느 플랫폼에서든 통하는 일반론이 본문이고, 특정 운영체제에 묶이는 구체적 설치법은 “플랫폼 노트” 상자로 격리해 두었다 — 상자는 이 책의 예시 플랫폼인 Windows 기준이며, 건너뛰어도 본문은 이어진다. 버그를 실행 중에 잡아 주는 그물(새니타이저)까지 장만하면 이 부가 끝난다.

돌아보기 14장에서 컴파일러에게 `-E`, `-S` 같은 부탁을 하며 명령줄에서 일했다. 요즘은 클릭으로 다 되는 세상인데, 왜 명령줄인가?

답 일반론이기 때문이다. 그래픽 개발 도구는 저마다 생김새가 다르고 몇 해 마다 바뀌지만, `cc hello.c -o hello`라는 문장은 반세기째 모든 플랫폼에서 통하는 공용어다. 그리고 어떤 그래픽 도구를 쓰든 그 “빌드” 버튼의 밑바닥에서 실제로 도는 것이 바로 이 명령들이다 — 밑바닥을 아는 사람은 도구를 갈아타도 길을 잃지 않는다. 이 책이 명령줄을 기준으로 삼는 이유다.

15.1 필요한 것은 셋뿐이다

개발환경이라는 말이 거창하지만, C 프로그래밍에 필요한 것은 셋뿐이다.

- **컴파일러** — 14장의 네 주자 일체. 이 책은 gcc와 clang 두 계열을 기준으로 한다(둘 다 무료이고 모든 주요 플랫폼에 있다).
- **텍스트 편집기** — 소스 코드는 그냥 텍스트 파일이므로(6장), 글자를 적을 수 있는 도구면 무엇이든 된다. 어떤 편집기를 쓰든가는 취향의 영역이라 이 책은 정하지 않는다.
- **터미널** — 명령을 적어 넣는 창. 모든 운영체제에 기본으로 있다.

일과의 모양도 단순하다 — **편집하고, 컴파일하고, 실행한다**. 이 세 박자의 반복이 프로그래밍의 기본 사이클이고, 이 책의 모든 예제가 이 사이클 위에서 돈다(물론 독자는 지면으로 읽기만 해도 된다 — 1장의 약속대로, 이 장의 설치조차 시연이지 숙제가 아니다).

▣ 플랫폼 노트 — Windows에서 컴파일러 설치하기

이 책의 예시 플랫폼은 Windows다. 두 갈래 중 하나(또는 둘 다)를 고르면 된다 — 이 책의 예제는 어느 쪽으로도 동일하게 빌드된다.

갈래 1 — LLVM clang (권장). LLVM 프로젝트의 공식 Windows 배포판을 설치한다. 터미널(PowerShell)에서 한 줄이면 된다:

```
> winget install LLVM.LLVM
```

(또는 llvm.org의 릴리스 페이지에서 설치 프로그램을 받아도 같다.) 설치 후 새 터미널에서 `clang --version`이 판 번호를 답하면 준비 끝 이다. 이후 이 책의 `cc` 자리에 `clang`을 쓰면 된다:

```
> clang hello.c -o hello.exe
> .\hello.exe
안녕, 세상!
```

갈래 2 — MinGW-w64 gcc. gcc의 Windows 이식판이다. MSYS2라는 꾸러미 관리 환경(msys2.org)을 설치한 뒤, 그 터미널에서:

```
$ pacman -S mingw-w64-ucrt-x86_64-gcc
$ gcc --version
```

셋째 길 — Visual Studio(MSVC)에 관하여. Microsoft의 자체 컴파일러와 통합 환경도 훌륭한 선택지이지만, 이 책의 명령·선택지 표기(gcc/clang 계열)와 달라서 여기서 다루지는 않는다. 관심 있는 독자는 Microsoft의 공식 문서(learn.microsoft.com/cpp)가 설치부터 첫 프로그램까지 자세히 안내하므로 그쪽을 따르면 된다.

문 컴파일러가 두 계열이나 되는 이유가 있는가? 하나만 있으면 헛갈릴 일도 없을 텐데.

답 경쟁이 곧 품질이기 때문이다 — 그리고 프로그래머에게는 공짜 검증 도구이기 때문이다. 같은 코드를 gcc와 clang 양쪽으로 컴파일해 보면, 한쪽이 놓친 경고를 다른 쪽이 잡아 주는 일이 흔하다. 두 독립적인 구현이 모두 통과시키는 코드는 표준(계약서)에 맞게 쓰였을 가능성이 훨씬 높다 — 10장의 표현을 빌리면, 방언이 아니라 표준어로 말하고 있다는 교차 증거다. 실제로 이 책의 수록 예제들도 두 컴파일러로 교차 검증한다.

15.2 경고 — 공짜 검토크를 켜 두어라

설치 직후에 들일 습관이 하나 있다. 컴파일러에게 **경고**를 넉넉히 켜 달라고 부탁하는 것이다:

```
$ cc -Wall -Wextra hello.c -o hello
```

-Wall -Wextra는 “수상한 구석을 최대한 알려 달라”는 뜻이다(이름과 달리 -Wall이 전부는 아니라서 관행상 둘을 함께 쓴다). 경고는 오류가 아니라 **컴파일러의 무료 코드 검토**다 — 계약(표준) 위반까지는 아니어도 사고 나기 좋은 무늬를 짚어 준다. 이 책의 모든 예제는 이 두 선택지를 켜 채, 경고 0개로 검증된다.

15.3 새니타이저 — 실행 중에 치는 그물

현대 C 개발환경의 필수 장비가 하나 더 있다. 컴파일 때가 아니라 **실행 중에** 사고를 잡는 도구, **새니타이저(sanitizer)**다.

원리는 이렇다 — 컴파일할 때 특별한 선택지를 주면, 컴파일러가 프로그램 곳곳에 **감시 코드**를 심어 준다. 그 프로그램을 실행하면 감시 코드가 메모리 접근과 연산을 지켜보다가, 잘못이 **일어나는 순간** 현장을 짚으며 프로그램을 세운다. 대표 선수는 셋이다.

- **ASan(AddressSanitizer)** — 메모리 사고 담당. 경계를 넘는 접근, 해제 한 기억을 다시 만지기 같은, 제7부에서 만날 위험들을 현장에서 잡는다.
- **UBSan(UndefinedBehaviorSanitizer)** — 계약 위반 담당. 부호 있는 오버플로(5장), 폭 이상 시프트(5장) 같은 **정의되지 않은 동작**이 실행 중에 실제로 일어나면 그 자리에서 알린다. 41장의 주역이다.
- **TSan(ThreadSanitizer)** — 경쟁 사고 담당. 여러 코어(10장)가 같은 데이터를 다투는 문제를 잡는다. 이 책의 범위 밖이지만 이름은 알아 둔다.

쓰는 법은 컴파일 선택지 하나다:

```
$ cc -fsanitize=address,undefined -g hello.c -o hello
$ ./hello
```

(-g는 “사고 지점을 소스 몇 번째 줄인지로 알려 달라”는 부가 정보 선택지다.) 이렇게 빌드한 프로그램은 조금 느려지는 대신, 사고를 조용히 지나치지 않는다 — 5장에서 본 “오버플로는 조용하다”에 대한 현대의 응답이 바로 이 도구들이다. 이 책의 뒷부분(제7부, 제9부)에서 위험한 코드를 다룰 때, 새니타이저가 사고를 잡아내는 장면을 지면에서 여러 번 보게 된다.

▣ 플랫폼 노트 — Windows에서 새니타이저 쓰기

플랫폼마다 지원 폭이 달라서, Windows의 사정을 정직하게 적어 둔다.

- **ASan·UBSan**: 갈래 1의 **clang**으로 쓸 수 있다 — 위의 `-fsanitize=address,undefined` 선택지가 그대로 통한다. 갈래 2의 MinGW-w64 gcc는 Windows에서 새니타이저를 지원하지 않는다 — 새니타이저를 쓰려면 clang을 고르면 된다(두 갈래를 다 설치해 두고 평소엔 아무 쪽, 검사할 땐 clang을 쓰는 것도 좋은 배치다).
- **TSan**: Windows 자체를 지원하지 않는다(리눅스·macOS 전용). Windows 에서 굳이 쓰려면 WSL(Windows 안의 리눅스 환경, Microsoft 공식 기능)을 통하는 길이 있다 — 다만 이 책의 범위에서는 필요하지 않다.

참고로 MSVC도 자체 ASan을 지원한다(`/fsanitize=address`) — 상세는 앞서 언급한 Microsoft 공식 문서에 있다.

문 경고도 켜고 새니타이저도 있는데, 그래도 버그가 남는 이유는 무엇인가?

답 각 도구가 보는 범위가 다르기 때문이다. 경고는 **컴파일 시점**에 코드의 생김새를 보고, 새니타이저는 **실행 시점**에 실제로 일어난 일을 본다 — 즉 실행해 보지 않은 경로의 사고는 새니타이저도 모른다. 그래서 현대적 C 개발은 그물을 겹친다: 경고(항상) + 새니타이저(시험 실행) + 두 컴파일러 교차(습관) + 그리고 애초에 사고 나기 어려운 부품을 쓰는 것. 마지막 항목이 이 책 후반의 proven이 서는 자리다(34장) — 도구는 그물이고, 좋은 부품은 애초에 떨어지지 않는 발판이다.

15.4 제3부를 닫으며

첫 프로그램을 읽었고(13장), 그것이 실행 파일이 되는 릴레이를 구경했으며(14장), 그 릴레이를 내 책상에서 돌릴 도구와 그물까지 갖췄다(15장). 이제 준비는 끝났다.

다음 부부터 본격적인 언어 공부다 — 단, 이 책의 방식대로 간다. 새 문법을 나열하는 것이 아니라, 13장의 헬로 월드 **한 편을 완전히 읽어 내는 것**이 제4부의 목표다. 프로그램의 뼈대(문장과 블록)부터, 수식, 함수를 부르는 법, 그리고 출력까지 — 이미 본 여섯 줄이 전부 “아는 문장”이 될 때까지.

제4부 — 최소한의 도구 상자

16 프로그램의 구조

집필 예정 (RFC-0002 rev.f 16장).

17 수식 — 값이 되는 것

집필 예정 (RFC-0002 rev.f 17장).

18 함수 사용 — 부르는 법

집필 예정 (RFC-0002 rev.f 18장).

19 출력

집필 예정 (RFC-0002 rev.f 19장).

제5부 — 선언: 이름을 만드는 법

20 변수 선언

집필 예정 (RFC-0002 rev.f 20장).

21 함수 선언과 정의

집필 예정 (RFC-0002 rev.f 21장).

22 입력

집필 예정 (RFC-0002 rev.f 22장).

제6부 — 값과 흐름

23 정수 — 유한한 수의 세계

집필 예정 (RFC-0002 rev.f 23장).

24 정수의 연산 — 나눗셈, 비트

집필 예정 (RFC-0002 rev.f 24장).

25 불리언과 비교

집필 예정 (RFC-0002 rev.f 25장).

26 결정 — if와 switch

집필 예정 (RFC-0002 rev.f 26장).

27 반복 — 루프와 불변식

집필 예정 (RFC-0002 rev.f 27장).

28 함수의 의미 — 값 복사와 부수효과

집필 예정 (RFC-0002 rev.f 28장).

제7부 — 기억

29 객체, 주소, 포인터

집필 예정 (RFC-0002 rev.f 29장).

30 널 — 삼형제의 정식 취급

집필 예정 (RFC-0002 rev.f 30장).

31 포인터의 규칙 — 정렬과 프로버넌스

집필 예정 (RFC-0002 rev.f 31장).

32 배열

집필 예정 (RFC-0002 rev.f 32장).

33 문자열

집필 예정 (RFC-0002 rev.f 33장).

34 안전한 입력과 proven의 등장

집필 예정 (RFC-0002 rev.f 34장).

35 수명과 저장 기간

집필 예정 (RFC-0002 rev.f 35장).

36 동적 메모리

집필 예정 (RFC-0002 rev.f 36장).

제8부 — 자료의 모양

37 구조체

집필 예정 (RFC-0002 rev.f 37장).

38 공용체와 표현

집필 예정 (RFC-0002 rev.f 38장).

제9부 — 정밀

39 실수 — 근사의 수학

집필 예정 (RFC-0002 rev.f 39장).

40 오류와 계약

집필 예정 (RFC-0002 rev.f 40장).

41 정의되지 않은 동작

집필 예정 (RFC-0002 rev.f 41장).

제10부 — 구성

42 여러 파일 — 분할과 링크

집필 예정 (RFC-0002 rev.f 42장).

43 표준 라이브러리의 지형

집필 예정 (RFC-0002 rev.f 43장).

44 proven 전면

집필 예정 (RFC-0002 rev.f 44장).

45 모던 C 총정리

집필 예정 (RFC-0002 rev.f 45장).